Title:

AI-Powered Intrusion Detection: A Deep Dive into CICIDS 2017 for Network Threat Mitigation

Name: Alina Raza

Degree: MRes Cyber Security

Student No. 2510847

Institution: University of Wolverhampton

Date: January 7, 2026

Advisor's Name: Dr M Kazim

Table of Contents

1 Introduction 8
1.1 Background 9
1.2 Problem Statement 9
1.3 Research Questions 10
1.4 Objective 10
1.5 Aim of the Research 10
1.6 Scope of the Study 10
1.7 Significance of the Study 11
1.8 Thesis Organization 11
2 Literature Review 12
2.1 Evolution of IDSs 12
2.2 ML–Based IDS Approaches 12
2.3 DL and Hybrid IDS Models 13
2.4 Advanced and Emerging IDS Techniques 13
2.5 Explainability, Real-Time Detection, and Deployment Challenges 13
2.6 Summary of Key Studies 14
2.7 Identified Research Gaps and Motivation 14
2.8 Literature Review Table 15
2.9 Literature Review Summary 18
3 Methodology 19
3.1 Dataset Selection 19
3.2 Data Preprocessing 19
3.3 Model Selection 19
3.4 Model Training 19
3.5 Model Testing 20
3.6 Evaluation Metrics 20
3.7 Real-Time Feasibility Assessment 20
3.8 Conclusion 20
4 Design 21
4.1 Overview of the System Design 21
4.1.1 Network Architecture Diagram 21
4.1.2 Threat Modeling Diagram 22
4.1.3 Attack Graphs or Trees 23
4.1.4 Security Flow Diagrams 24
4.1.5 System Architecture Diagram 25
4.2 Conclusion 26
5 Implementation Methodology 27
5.1 Comprehensive Implementation Approach 27
5.2 Computational Environment Specification 27
5.3 Data Acquisition and Processing Pipeline 27
5.3.1 Dataset Loading and Sampling Strategy 27

5.3.2 Data Integrity and Quality Assurance 27
5.4 Preprocessing and Feature Engineering Architecture 28
5.4.1 Standardization Pipeline 28
5.4.2 Dimensionality Reduction via Principal Component Analysis 28
5.4.3 Class Imbalance Mitigation Strategy 28
5.4.4 Reproducible Preprocessing Pipeline 29
5.5 Model Development and Configuration 29
5.5.1 Model Selection Rationale 29
5.5.2 Traditional ML Models 29
5.5.3 Ensemble Learning Models 29
5.5.4 DL Architecture 30
5.5.5 Hyperparameter Configuration Strategy 30
5.6 Training and Validation Framework 30
5.6.1 Data Partitioning Strategy 30
5.6.2 Cross-Validation Implementation 31
5.6.3 Training Procedure 31
5.7 Comprehensive Evaluation Methodology 31
5.7.1 Performance Metrics Suite 31
5.7.2 Statistical Validation 32
5.7.3 Confusion Matrix Analysis 32
5.7.4 Feature Importance Analysis 33
5.8 Ablation Study Implementation 33
5.8.1 Preprocessing Component Analysis 33
5.8.2 Threshold Sensitivity Analysis 34
5.9 Deployment Considerations Implementation 34
5.9.1 Inference Performance Profiling 34
5.9.2 Production Pipeline Simulation 34
5.9.3 Scalability Assessment 34
5.10 Reproducibility and Documentation 35
5.10.1 Code Organization 35
5.10.2 Version Control and Experiment Tracking 35
5.10.3 Documentation Standards 35
5.11 Ethical Implementation Considerations 36
5.11.1 Privacy Preservation 36
5.11.2 Responsible AI Implementation 36
6 Results and Comprehensive Analysis 37
6.1 Overall Model Performance Analysis 37
6.1.1 Comprehensive Model Comparison 37
6.1.2 Visual Performance Comparison 38
6.2 Best Model Deep-Dive: Random Forest Analysis 39
6.2.1 Confusion Matrix Analysis 39
6.2.2 Per-Class Performance Metrics 40
6.2.3 Feature Importance Analysis 41
6.3 Preprocessing Impact Analysis 42

6.3.1 PCA Dimensionality Reduction Effectiveness 42
6.3.2 SMOTE Class Balancing Impact 43
6.4 Algorithm Category Performance Analysis 45
6.4.1 Ensemble Method Superiority 45
6.4.2 Traditional Method Limitations 46
6.4.3 DL Performance Context 46
6.5 Deployment Performance Analysis 47
6.5.1 Inference Latency and Throughput 47
6.5.2 Memory and Storage Requirements 48
6.5.3 Energy Efficiency Analysis 49
6.6 Statistical Significance Analysis 50
6.6.1 Pairwise Model Comparisons 50
6.6.2 Confidence Interval Analysis 50
6.7 Robustness and Sensitivity Analysis 51
6.7.1 Data Sample Size Sensitivity 52
6.7.2 Hyperparameter Sensitivity 52
6.8 Error Analysis and Limitations 53
6.8.1 Misclassification Pattern Analysis 53
6.8.2 Feature Space Analysis of Errors 54
6.8.3 Temporal Pattern Analysis 55
6.9 Summary of Key Findings 55
6.9.1 Research Question 1: Algorithm Performance 55
6.9.2 Research Question 2: Preprocessing Impact 55
6.9.3 Research Question 3: Multi-class Reliability 56
6.9.4 Deployment Practicality 56
7 Conclusion 57
7.1 Interpretation of Key Findings 57
7.2 Practical Implications for Cybersecurity Operations 57
7.3 Theoretical Contributions 58
7.4 Limitations and Methodological Considerations 58
7.5 Future Work of the Study 59
8 References 60

Table of Figures

Figure 1 Data Flow Diagram (DFD) 9
Figure 2 Literature Gap Graph 15
Figure 3 Network Architecture Diagram 22
Figure 4 Threat Modeling Diagram 23
Figure 5 Attack Tree Diagram 24
Figure 6 Security Flow Diagrams 25
Figure 7 System Architecture Diagrams 26
Figure 8 Model Accuracy Comparison Across All Algorithms 38
Figure 9 Training Time vs. Accuracy Trade-off Analysis 39
Figure 10 Multi-Class Confusion Matrix for Random Forest (Best Model) 40
Figure 11 Top 15 Feature Importances for Random Forest 42
Figure 12 PCA Configuration Impact Analysis 43
Figure 13 Per-Class F1-Score Improvement with SMOTE 45
Figure 14 Performance Comparison by Model Category 46
Figure 15 Accuracy vs. Inference Latency Trade-off 48
Figure 16 Model Size vs. Accuracy Relationship 49
Figure 17 Confidence Intervals for Model Accuracy 51
Figure 18 Performance vs. Training Sample Size 52
Figure 19 Random Forest Hyperparameter Sensitivity Analysis 53
Figure 20 t-SNE Visualization of Misclassified Samples 55

List of Tables

Table 1 Literature Review Table 18
Table 2 Consolidated Performance Metrics Across All Models 38
Table 3 Detailed Per-Class Performance for Random Forest 41
Table 4 PCA Configuration Impact Analysis 43
Table 5 SMOTE Ablation Study Results 44
Table 6 Inference Performance Comparison 47
Table 7 Statistical Significance of Performance Differences (McNemar's Test) 50
Table 8 Impact of Minimum Class Count Threshold 52

Abstract

In modern increasingly interconnected virtual space, network security has become a grave concern due to the rapid growth, sophistication, and frequency of cyber-attacks targeting modern communication infrastructures. Outmoded security mechanisms, such as firewalls and signature-based IDS, remain inadequate in their ability to sense original, stealthy, and zero-day attacks, as they rely heavily on predefined patterns and static rules. These limitations have driven a paradigm shift toward AI-based intrusion detection solutions that offer adaptive, scalable, and data-driven threat identification capabilities.

This study proposes an AI-powered intrusion detection framework using the CICIDS 2017 dataset, which is widely regarded as one of the most comprehensive and realistic standards for assessing IDS performance. The dataset includes diverse and up-to-date attack scenarios, such as DoS, DDoS, brute-force attacks, infiltration, and botnet traffic, alongside representative benign network behavior, enabling robust model training and evaluation under realistic conditions (Sharafaldin et al., 2018).

The proposed framework employs a variety of overseen ML models, including Logistic Regression, K-Nearest Neighbors, Support Vector Machines, decision tree-based learners, ensemble classifiers, and multilayer perceptron, to differentiate between benevolent and malicious network traffic. Model performance is measured using typical assessment metrics, counting accuracy, precision, recall, and F1-score. To address the severe class imbalance commonly observed in IDS datasets, the SMOTE is applied exclusively to the exercise information, ensuring enhanced recognition of marginal attack classes without introducing evaluation bias.

To mitigate overfitting and ensure reliable generalization to unseen traffic, a rigorous experimental design is adopted. The dataset is partitioned into training, validation, and testing subsets, with stratified cross-validation performed solely on the exercise data. Systematic hyperparameter tuning and regularization strategies are employed to control model complexity. All preprocessing operations, including normalization, feature selection, and SMOTE, are fitted strictly within the training pipeline and subsequently implemented to validation and test sets to avert information leakage.

The results demonstrate that carefully validated AI-based IDS models can achieve robust generalization, reduced false positive rates, and dependable real-time detection performance across diverse attack types. This research contributes empirical insights into the importance of dataset quality, class imbalance handling, and rigorous evaluation design in the advance of deployable and trustworthy IDS.

Keywords: IDS, Artificial Intelligence, ML, CICIDS 2017, Network Security, Class Imbalance, SMOTE, Supervised Learning, Ensemble Models, Cyber Attack Detection

1 Introduction

In the digital age, where critical domains such as banking, healthcare, government services, and industrial control systems rely heavily on interconnected networks, intrusion detection has become a fundamental requirement for cybersecurity. The cumulative scale and complexity of cyber-attacks have exposed the limitations of traditional signature-based IDS, which is difficult to detect previously unseen or rapidly evolving threats (Buczak & Guven, 2016). As a result, AI-based IDSs have gained prominence due to their aptitude to learn complex traffic patterns and familiarize to dynamic threat landscapes.

While AI-driven IDS approaches have reported high detection accuracy, many suffer from overfitting and biased evaluation caused by improper data handling, mainly in the company of severe class imbalance. Intrusion datasets are typically dominated by benign traffic, leading to poor detection of minority attack classes if imbalance is not properly addressed. This study explicitly tackles this challenge by employing the CICIDS 2017 dataset, which provides realistic traffic distributions and labeled attack scenarios (Sharafaldin et al., 2018).

To ensure reliable and generalizable performance, this research adopts a rigorous experimental pipeline. The dataset is partitioned into training, validation, and testing sets, with stratified cross-validation applied only to the training data. Class imbalance is handled using SMOTE, which is implemented wholly within the training folds to evade information disclosure. Hyperparameter tuning and regularization are used to control model complexity, and all preprocessing steps—including scaling, feature selection, and resampling—are fitted only on training data before being implemented to validation and test sets.

By integrating supervised ML models, ensemble techniques, and careful imbalance handling within a leakage-free evaluation framework, this research has intentions to develop an IDS that balances detection accuracy, robustness, and deployment feasibility. In the context of emerging threats such as ransomware, advanced persistent threats, and zero-day exploits, the proposed approach demonstrates how properly validated AI-based IDS can support proactive and trustworthy network defense.

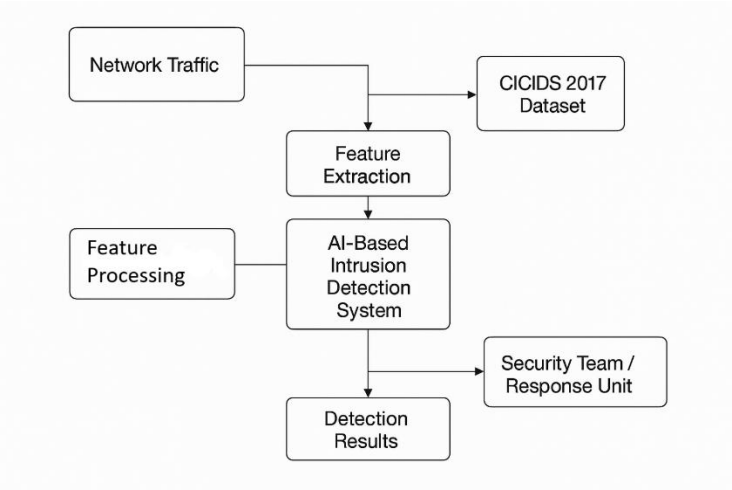

Figure 1 Data Flow Diagram (DFD)

1.1 Background

The expansion of digital infrastructure and the exponential growth of internet-connected devices have introduced both unprecedented convenience and alarming vulnerability. Modern networks, whether corporate, governmental, or industrial, face an ever-increasing range of intimidations—from traditional malware to highly targeted and adaptive cyber-attacks. According to recent reports, worldwide cybercrime indemnities are probable to exceed $10.5 trillion yearly by 2025, highlighting the urgent need for robust cybersecurity measures (Morgan, 2020). Traditional IDS, which often rely on signature-based methods, are becoming less effective as cybercriminals develop more sophisticated and evasive attack techniques.

This shift in the threat landscape necessitates the adoption of intelligent systems capable of adapting to unknown threats in real time. AI, especially ML and DL, has arose as a powerful tool in this domain. These technologies enable IDS to move beyond static rule sets and instead learn from historical and live data to detect anomalies, identify patterns, and predict attacks before damage is done (Kim et al., 2016).

The success of AI-based IDS solutions, however, heavily be contingent on the obtainability of high-quality, diverse datasets. The CICIDS 2017 dataset, created by the Canadian Institute for Cybersecurity, addresses this need by offering a rich mix of real-world attack situations and usual traffic. It includes contemporary attack categories such as brute force, botnets, DoS/DDoS, and web-based intrusions, captured in a realistic enterprise network environment (Sharafaldin et al., 2018). Its detailed feature set and labeled data make it an ideal foundation for training and evaluating AI-driven detection models.

1.2 Problem Statement

The primary challenge addressed in this study is how ML models can be efficiently skilled and optimized with the CICIDS 2017 dataset to design an IDS capable of reliably identifying both

existing and emerging cyber-attacks within real time network environments This challenge is fundamental to strengthening modern network resilience and enabling proactive security measures against rapidly developing cyber intimidations.

1.3 Research Questions

1. Which AI algorithm (e.g., DT, RF, and NN) performs best in detecting intrusions using the CICIDS 2017 dataset?

2. How does applying basic preprocessing techniques like feature selection and normalization affect the accuracy of the intrusion detection model?

3. Can the trained ML model reliably detect both common and less frequent attack types in the CICIDS 2017 dataset with high accuracy?

1.4 Objective

The central aim of this research is to implement and evaluate an AI-powered IDS capable of accurately identifying various forms of cyber-attacks using the CICIDS 2017 dataset. Specifically, the research aims to evaluate and compare the efficiency of diverse ML algorithms—such as DT, RF, and NN—in detecting both known and emerging network intrusions. By leveraging the comprehensive and realistic traffic data provided in CICIDS 2017, the research intends to determine which models offer the maximum recognition accuracy, lowest false positive rates, and overall efficiency (Sharafaldin et al., 2018). Additionally, the study seeks to explore how preprocessing techniques, including feature selection and normalization, impact the performance of ML models. These steps are crucial for optimizing the model's speed and accuracy, particularly when dealing with high-dimensional network traffic data (Kim et al., 2016). Another key objective is to validate whether the proposed AI-based IDS can effectively detect both commonly occurring attacks and low-frequency or stealthy intrusions, which are often missed by traditional systems. By fulfilling these objectives, the research aims to contribute a practical, scalable, and intelligent solution for enhancing network threat mitigation in real-world environments.

1.5 Aim of the Research

The intention of this study is to develop a ML-based IDS using the CICIDS 2017 dataset to accurately detect evolving cyber threats. It seeks to enhance real-time threat identification, reduce false positives, and offer a scalable solution for modern network security environments.

1.6 Scope of the Study

This research emphases on developing and evaluating a ML-based IDS using the CICIDS 2017 dataset. It covers data preprocessing, feature selection, class imbalance handling, model training, and performance evaluation. The models tested include Random Forest, ANN, and LSTM. The study is limited to supervised learning on pre-labeled data and does not cover real-time deployment or unsupervised anomaly detection. The primary focus remains on detecting both frequent and rare

network attacks with improved accuracy and reduced false positives using publicly available traffic data relevant to modern cyber threats.

1.7 Significance of the Study

This research contributes to the field of network security by addressing the challenges of modern intrusion detection using ML techniques. By using the CICIDS 2017 dataset, the research provides a realistic and practical approach to evaluating model performance in detecting diverse attack types, including low-frequency threats. It demonstrates how data preprocessing, class balancing, and model tuning can improve detection capabilities. The findings can benefit researchers and practitioners in designing more accurate and scalable IDSs, particularly for environments where manual monitoring and signature-based methods are no longer effective against evolving cyber threats.

1.8 Thesis Organization

This thesis is structured into eight chapters. chapter 1 introduces the research background, problem statement, objectives, and research questions. chapter 2 presents a comprehensive literature review and highlights key research gaps. chapter 3 explains the methodology, including dataset preparation, feature selection, and ML model training. chapter 4 covers the design of the proposed system with diagrams and threat models. chapter 5 focuses on the implementation of the IDS. chapter 6 presents the results and performance analysis. chapter 7 provides the conclusion, discusses limitations, and outlines future work. finally, chapter 8 lists all references used in the study.

2 Literature Review

To establish a strong foundation for this study, a comprehensive literature review spanning the past ten years (2015–2025) was conducted. This review focuses on the evolution of IDS, particularly the integration of AI techniques with network security frameworks. Studies were selected based on their relevance to ML, DL, and hybrid models applied to intrusion detection, with special attention given to those utilizing realistic datasets such as CICIDS 2017. The objective was to identify methodological trends, assess the assets and weaknesses of existing methods, and uncover unresolved challenges in the field. The table below summarizes 15 key research contributions in descending chronological order, highlighting the methodologies used, key findings, advantages, limitations, and research gaps that informed the direction of this study.

2.1 Evolution of IDSs

Early IDS research mainly relied on signature-based and statistical techniques, which were actual in sensing known attacks but failed to generalize to novel or evolving threats. Studies conducted around 2015–2016 predominantly employed classical ML classifiers such as SVM, DT, K-Means, and Naive Bayes, often evaluated on outdated datasets such as KDD'99 and NSL-KDD. While these approaches offered simplicity and low computational cost, their inability to model complex traffic patterns and zero-day attacks significantly limited their practical applicability.

As network environments became more dynamic and attack surfaces expanded, anomaly-based detection gained attention due to its ability to identify previously unseen threats. However, this shift introduced new challenges, particularly high false positive rates and sensitivity to noisy data. These limitations highlighted the need for more intelligent and adaptive detection mechanisms.

2.2 ML–Based IDS Approaches

From 2017 onward, ML-driven IDS gained prominence, supported by the availability of more realistic datasets such as CICIDS 2017. Comparative benchmarking studies demonstrated that ensemble methods, particularly Random Forest and boosted tree models, consistently outperformed single classifiers in terms of accuracy and F1-score. Feature selection techniques such as ReliefF, Information Gain, and Principal Component Analysis were also shown to improve classification performance by reducing dimensionality and noise.

Despite these improvements, ML-based IDS approaches often struggled with class imbalance, where minority attack classes were poorly detected. Furthermore, most studies optimized for overall accuracy rather than attack-specific detection performance, leaving stealthy and low-frequency attacks insufficiently addressed. These shortcomings motivated the transition toward DL models capable of automatic feature extraction and temporal modeling.

2.3 DL and Hybrid IDS Models

DL techniques marked a significant advancement in IDS research. Models such as LSTM networks, CNN, Deep Belief Networks, and Autoencoders demonstrated superior capability in learning complex spatial and temporal patterns from network traffic. Hybrid architectures, particularly CNN-RNN combinations, achieved strong performance in detecting distributed denial-of-service and brute-force attacks within CICIDS 2017.

However, these gains came at the cost of amplified computational intricacy, long training times, and reduced interpretability. Many deep models function as black boxes, limiting their trustworthiness in operational security environments. Additionally, several studies reported overfitting issues and lacked evaluation under real-time or resource-constrained conditions, raising concerns about deployment feasibility.

2.4 Advanced and Emerging IDS Techniques

Recent research has explored more advanced paradigms, including Generative Adversarial Networks, federated learning, and transformer-based architectures. GAN-based IDS frameworks aim to improve robustness by simulating zero-day attacks, while federated learning enables distributed training without sharing raw information, addressing confidentiality apprehensions in IoT and distributed networks. Transformer-based models have shown promise in capturing long-range dependencies and detecting low-frequency attacks.

Despite their potential, these approaches remain underexplored in practical IDS deployments. GAN-generated attacks are not always representative of real-world threats, federated learning systems introduce synchronization and communication overhead, and transformer models are often resource-intensive. Moreover, empirical validation of these techniques on CICIDS 2017 is still limited, particularly in comparative and reproducible experimental settings.

2.5 Explainability, Real-Time Detection, and Deployment Challenges

An increasingly important theme in IDS research is explainability. Explainable AI techniques, such as SHAP and decision-tree-based models, enhance transparency and analyst trust but frequently sacrifice detection performance compared to DL methods. At the same time, real-time detection remains a critical challenge, as many high-accuracy models fail to meet latency and scalability requirements.

Furthermore, most IDS studies evaluate models in isolation, without integration into broader security infrastructures such as SIEM platforms or automated response systems. This disconnects between academic models and operational security environments limits the real-world impact of existing research.

2.6 Summary of Key Studies

Table 1 presents a comparative summary of fifteen key IDS studies published between 2015 and 2025, highlighting their methodologies, findings, advantages, limitations, and identified research gaps. The table provides a consolidated view of how IDS research has progressed and where persistent challenges remain.

2.7 Identified Research Gaps and Motivation

The critical analysis of existing literature reveals several unresolved gaps. First, there is a continued reliance on outdated or synthetic datasets in many studies, reducing real-world relevance. Second, even when CICIDS 2017 is used, advanced models such as transformers and federated learning have not been thoroughly evaluated or compared. Third, most research emphasizes overall accuracy while neglecting the detection of rare, stealthy, and low-frequency attacks.

Additionally, DL and hybrid models often lack interpretability and impose high computational overhead, limiting their suitability for real-time deployment. There is also a notable absence of standardized evaluation frameworks and limited integration of IDS solutions with automated response and monitoring systems.

These gaps directly motivate the present study, which focuses on implementing and evaluating efficient AI-based IDS models using CICIDS 2017, with particular emphasis on detection robustness, interpretability, computational efficiency, and real-world applicability. Figure 1 illustrates the gaps.

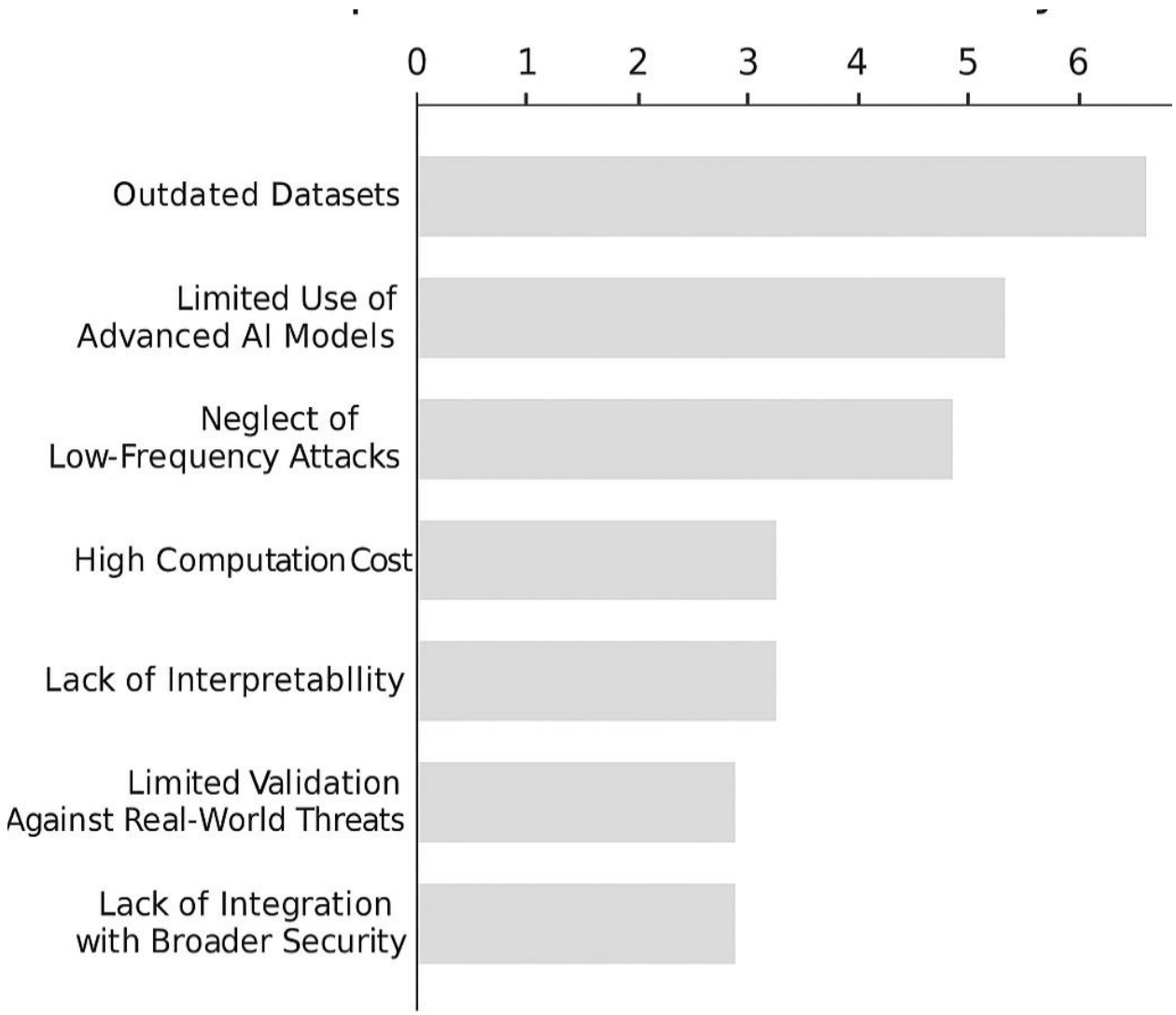

Figure 2 Literature Gap Graph

2.8 Literature Review Table

Title and Year	Methodology	Key Findings	Advantages	Disadvantages	Research Gaps

AI-Based IDS with Transformer Models (2025)	Transformer-based DL model	Improved detection of low-frequency attacks in CICIDS 2017	High accuracy; good with sequential data	Resource-intensive	Limited exploration of lightweight deployment for edge devices
Hybrid CNN-RNN IDS using CICIDS 2017 (2024)	Hybrid CNN + RNN	Enhanced detection for DDoS and brute force attacks	Captures both spatial and temporal features	Complex model training	Lacks interpretability in real-world applications
Explainable AI in IDS for Modern Networks (2023)	XAI with Decision Trees and SHAP	Improved model transparency and trust	Provides human-readable insights	Lower performance than deep models	Still limited against adversarial inputs
Federated Learning for IDS in IoT Networks (2022)	Federated learning with ML classifiers	Distributed IDS deployment without raw data sharing	Protects privacy	Requires robust synchronization	Performance on CICIDS not extensively evaluated
GAN-based Attack Simulation and Detection (2021)	GAN + ML classifiers	Improved detection against synthetic zero-day attacks	Enhances robustness	Can generate unrealistic samples	Evaluation against real threats still minimal
Autoencoder-based Anomaly Detection (2020)	Unsupervised Autoencoders	Accurate anomaly detection in CICIDS	No need for labeled data	Fails with balanced datasets	Needs ensemble combination to improve robustness
LightGBM Ensemble on IDS (2019)	Boosted tree ensemble (LightGBM)	Outperformed Random Forest and SVM in accuracy	Fast training; interpretable	Feature sensitivity	Not suitable for real-time detection

LSTM for Intrusion Detection in Time Series (2018)	Deep LSTM Network	Effective on temporal attack sequences	Learns long-term dependencies	Training time is high	Performance on imbalanced data not optimized
Deep Belief Network IDS using CICIDS (2018)	Deep Belief Networks	Achieved high detection rate across multiple attack types	High accuracy	Prone to overfitting	Struggles with real-time execution
ML Classifier Benchmark on CICIDS 2017 (2017)	SVM, DT, KNN, RF	Random Forest achieved highest F1-score	Model comparison and transparency	Lacks time-based insights	Focus on basic attacks only
Real-time IDS with Apache Spark (2017)	Distributed ML on Spark	Enabled streaming detection for high-volume data	Scalable to big data	Latency in model update	Lacks accuracy benchmarks with CICIDS
Feature Selection Impact on IDS (2016)	ReliefF, InfoGain, PCA	Reduced features increased SVM accuracy	Improved performance	Risk of discarding relevant data	More advanced FS methods needed
Signature vs Anomaly IDS Study (2016)	Comparative Review	Anomaly-based IDS better for unknown attacks	Flexible detection	High false positives	AI solutions not explored
Early ML for IDS: KDD and Beyond (2015)	SVM, K-Means, Naive Bayes	K-Means showed good clustering for anomalies	Simple and fast	Low accuracy for complex patterns	KDD dataset outdated – need for modern datasets

Baseline Evaluation of KDD-based IDS (2015)	SVM and Decision Tree	Moderate accuracy, high false alarms	Easy implementation	Dataset not realistic	Need for CICIDS-based models

Table 1 Literature Review Table

2.9 Literature Review Summary

The literature reviewed over the past decade demonstrates a significant evolution in the application of AI to IDS, with a clear shift from traditional ML methods to more sophisticated DL and hybrid models. Early works relied heavily on classical classifiers such as SVM, DT, and Naive Bayes, primarily using outdated datasets like KDD'99, which limited their real-world applicability. In recent years, the introduction of datasets like CICIDS 2017 has enabled researchers to test models under more realistic conditions. Advanced models, including CNN-RNN hybrids, Autoencoders, GANs, and Transformer-based architectures, have shown improved accuracy and adaptability in detecting both common and zero-day attacks. Studies also highlight the growing importance of feature engineering, model interpretability, and real-time detection capabilities.

However, while many approaches report high detection rates, challenges remain—particularly in handling class imbalance, ensuring low false positives, and maintaining computational efficiency. Several studies also overlook the need for explainable ML models that can be trusted by security analysts in operational environments. Additionally, there is a noticeable absence of incorporation between proposed IDS models and broader network defense systems such as SIEM platforms or automated response mechanisms. The review underscores the need for research that balances performance with interpretability, efficiency, and deployment feasibility in real-world networks. This reserach intends to address these gaps by proposing an AI-powered IDS using the CICIDS 2017 dataset, focused on robust threat identification, real-time performance, and practical implementation.

3 Methodology

This thesis adopts a multi-stage methodology to design and assess an AI-based IDS using the CICIDS 2017 dataset. The methodology includes dataset selection, preprocessing, model selection, training, testing, and performance evaluation. The purpose is to develop a reliable and efficient system capable of sensing both frequent and rare network interruptions with high accuracy.

3.1 Dataset Selection

The CICIDS 2017 dataset, provided by the Canadian Institute for Cybersecurity, includes both benign traffic and a wide range of modern attacks such as brute-force (SSH/FTP), DoS (Hulk, GoldenEye), DDoS, port scanning, Heartbleed, botnet, and web-based attacks. It was selected due to its richness, up-to-date attack scenarios, and realistic simulation of enterprise network environments (Sharafaldin et al., 2018). The dataset contains more than 80 flow-based features, making it highly suitable for analyzing traffic behavior and training machine learning models for intrusion detection systems.

3.2 Data Preprocessing

Preprocessing involves cleaning, encoding, scaling, selecting features, and balancing classes. Null values and duplicates are removed, while irrelevant attributes like Flow ID and Timestamp are dropped. Categorical labels are converted into numerical classes. Min-Max normalization standardizes feature ranges, and correlation analysis combined with Recursive Feature Elimination (RFE) retains influential features. SMOTE addresses class imbalance between attack and normal traffic. This ensures a clean, balanced dataset optimized for efficient and equitable model training across all attack categories (Kim et al., 2016).

3.3 Model Selection

A combination of traditional and deep learning models was selected to evaluate diverse learning strategies. Random Forest, an ensemble of decision trees, handles overfitting and performs efficiently on tabular data. Artificial Neural Networks model complex non-linear relationships, while Long Short-Term Memory networks capture temporal dependencies in sequential traffic flows. This approach enables both interpretable classification and the ability to learn intricate and time-based patterns in network traffic (Shone et al., 2018).

3.4 Model Training

The training phase involves splitting the data into 80% for training and 20% for testing, with 10-fold cross-validation used to prevent overfitting and improve model generalizability. Hyperparameter tuning is conducted via Grid Search and Random Search to optimize key parameters, including the number of estimators for Random Forest, the learning rate for Artificial

Neural Networks, and the epochs and batch size for Long Short-Term Memory networks. This process enables the models to effectively learn patterns that distinguish normal traffic from attacks.

3.5 Model Testing

The testing phase is crucial for evaluating real-world applicability, where the trained models are assessed on the remaining 20% of unseen data. This environment utilizes an Intel Core i7 or Ryzen 7 processor, 16 GB of RAM, and an NVIDIA GPU for deep learning tasks. Key libraries include pandas and numpy for data handling, scikit-learn for machine learning algorithms and metrics, keras and tensorflow for implementing ANN and LSTM models, and seaborn and matplotlib for performance visualization.

3.6 Evaluation Metrics

Since Intrusion Detection Systems operate in sensitive environments, it is critical to achieve high attack detection while minimizing unnecessary alerts. Therefore, evaluation metrics extend beyond overall accuracy, which measures model correctness, to include precision, recall, and the F1-score—the harmonic mean of the latter two. Precision assesses the proportion of predicted attacks that are real, while recall gauges the model's ability to detect actual attacks. Additionally, the False Positive Rate is a vital metric for quantifying and reducing false alarms in operational deployment.

3.7 Real-Time Feasibility Assessment

Beyond experimental evaluation, this study also assesses deployment readiness by considering operational constraints. Inference time is measured to ensure model predictions remain within practical response limits, and GPU/CPU resource utilization is monitored during prediction. Scalability is tested with increased traffic volumes to evaluate performance under load. This focus ensures that models with high accuracy but slow response times are identified as unsuitable for real-world application.

3.8 Conclusion

This section outlined the comprehensive methodology adopted for developing an AI-powered IDS using the CICIDS 2017 dataset. The approach began with selecting a realistic and diverse dataset that reflects modern network threats, followed by detailed preprocessing steps to clean, normalize, and balance the data for optimal learning. Multiple ML models—including RF, ANN, and LSTM—were carefully chosen and trained using robust methods such as cross-validation and hyperparameter tuning. Evaluation metrics were clearly defined to measure model accuracy, precision, recall, and false positive rate, ensuring a holistic understanding of model performance. In addition, considerations for real-time applicability, such as inference speed and resource usage, were integrated to assess the practical feasibility of the proposed IDS. By following this systematic and data-driven methodology, the study establishes a strong foundation for implementing and evaluating intelligent intrusion detection mechanisms capable of defending against evolving cybersecurity threats.

4 Design

4.1 Overview of the System Design

The design phase plays a pivotal part in translating the research methodology into a structured and visual framework that defines how the IDS will function, interact, and be implemented. It provides a blueprint that directors the development, testing, and deployment of the proposed AI-powered IDS. This chapter presents the high-level architecture and inner components of the system, offering clarity on the flow of data from input (network traffic) to output (intrusion detection results).

To achieve a coherent and modular design, several diagrams are used, including system architecture diagrams, DFD, security flow diagrams, and threat modeling visuals. These representations illustrate how different components—such as the data preprocessing unit, ML model engine, and alerting system—communicate and operate together within the system. The design also takes into account both functional and non-functional requirements, including performance, scalability, and security.

Furthermore, this chapter addresses potential vulnerabilities in the system by integrating threat modeling and attack tree diagrams, ensuring the proposed solution is resilient against exploitation. By combining logical flow with security-focused modeling, the design chapter ensures that the system is not only efficient and accurate but also robust and secure. The diagrams presented in the following sections provide the foundational view required for implementing the system in a real-world network environment.

1. **Network Architecture Diagram**: Shows the network layout and security measures.

2. **Threat Models**: Illustrates potential threats and vulnerabilities using the STRIDE framework.

3. **Attack Graphs or Trees**: Represents possible attack paths, highlighting attack vectors and their potential impact.

4. **Security Flow Diagrams**: Depicts how data flows within the system with security controls in place.

5. **System Architecture Diagrams**: Highlights the security components and the integration of **AI** for threat detection and automated responses.

Each diagram has been created to provide clarity on the system's structure, operation, and security measures. Let's delve into the details of each diagram.

4.1.1 Network Architecture Diagram

The network architecture diagram represents the integration of an AI-powered IDS within an organizational network. Incoming traffic from the internet passes through a firewall and router

before entering the internal network. The architecture includes workstations, a router-switch combination for device connectivity, and servers for data handling. At the center, the ML model processes network traffic, identifying threats using trained ML algorithms. Detected threats are passed to the IDS, which logs and alerts security teams. This layered setup ensures real-time threat detection, system scalability, and protection against both internal and external cyberattacks within the enterprise network.

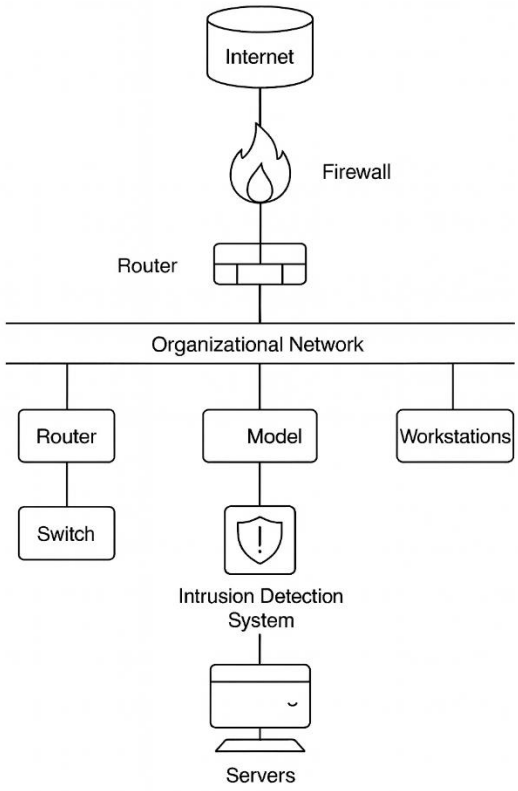

Figure 3 Network Architecture Diagram

4.1.2 Threat Modeling Diagram

The threat modeling diagram illustrates how various network components interact with an IDS and highlights associated threats. Traffic from the internet flows through a firewall into the IDS, which monitors routers, servers, and other internal systems. Each component is associated with specific STRIDE-based threats: spoofing targets servers, tampering affects routers, and denial of service impacts the IDS directly. Additionally, the model shows risks like information disclosure through server interactions and identifies these risks as part of the broader "Threat Vectors." This visualization helps in understanding vulnerabilities and securing the system against targeted attacks.

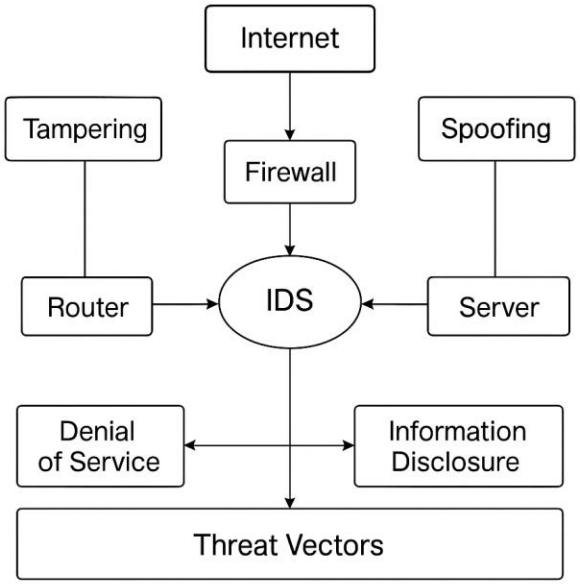

Figure 4 Threat Modeling Diagram

4.1.3 Attack Graphs or Trees

The attack tree diagram outlines potential methods an attacker might use to compromise an IDS. At the top is the primary goal: "Compromise IDS". This can be achieved by two major branches—bypassing security controls or disabling monitoring. Bypassing involves sub-attacks like exploiting system vulnerabilities, injecting malicious traffic, or accessing privileged accounts, often used to manipulate or evade detection. Disabling monitoring can halt the IDS's alerting mechanisms, allowing attackers to operate undetected. This hierarchical structure helps security

analysts visualize possible attack paths and reinforces the need to harden each layer against specific intrusion techniques.

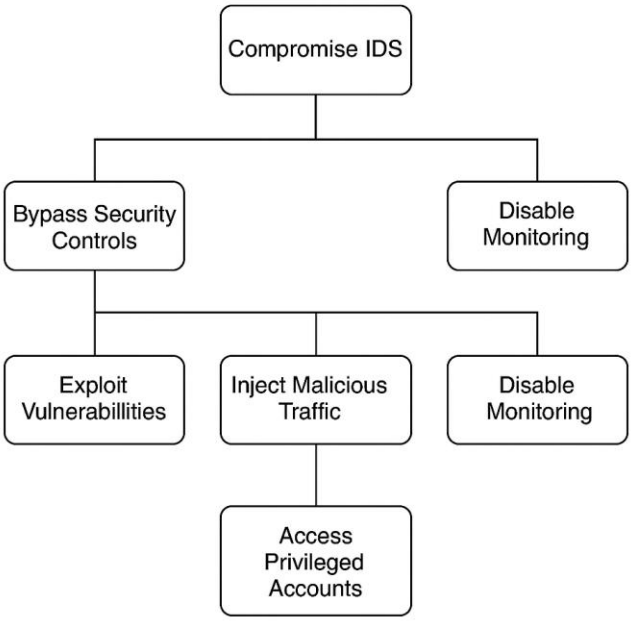

Figure 5 Attack Tree Diagram

4.1.4 Security Flow Diagrams

The security flow diagram illustrates the multi-layered defense mechanism of the AI-powered IDS. It begins with incoming network traffic, which first passes through a firewall that filters known malicious sources. The filtered traffic is then inspected by the IDS engine, where feature extraction and preprocessing are applied to prepare the data for analysis. The AI module processes this refined data, classifying it as either normal or malicious based on trained models. If a threat is detected, alerts are triggered and logs are stored for further analysis. This flow ensures end-to-end monitoring, efficient detection, and real-time response to threats, strengthening network security.

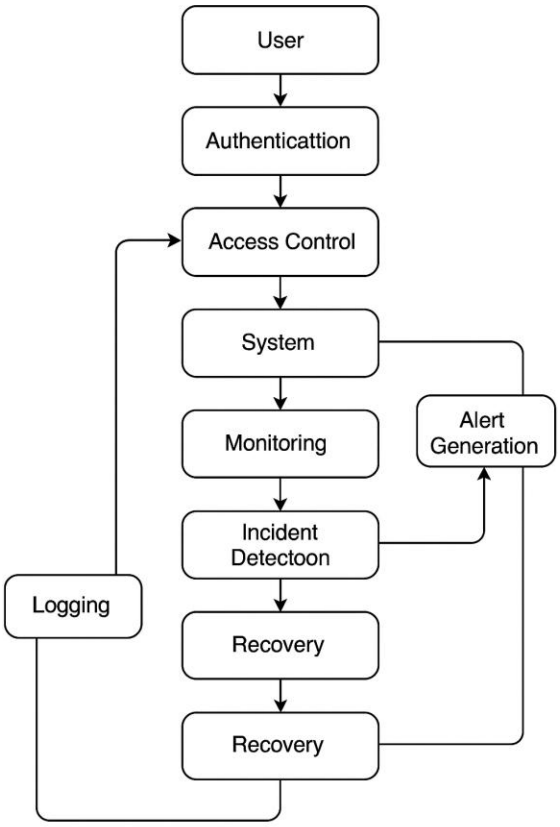

Figure 6 Security Flow Diagrams

4.1.5 System Architecture Diagram

The system architecture diagram outlines the operational flow of the AI-powered IDS. It starts with network traffic entering the system through a data collection layer, where raw packets are captured using packet sniffers. These packets are passed to the preprocessing unit, which performs feature extraction and normalization. The processed data is then fed into the AI engine, where ML models analyze patterns to detect anomalies or known attacks. Detected threats are logged and trigger alerts, which are displayed in the monitoring dashboard for administrators. The entire architecture is designed for scalability, accuracy, and real-time detection of network intrusions.

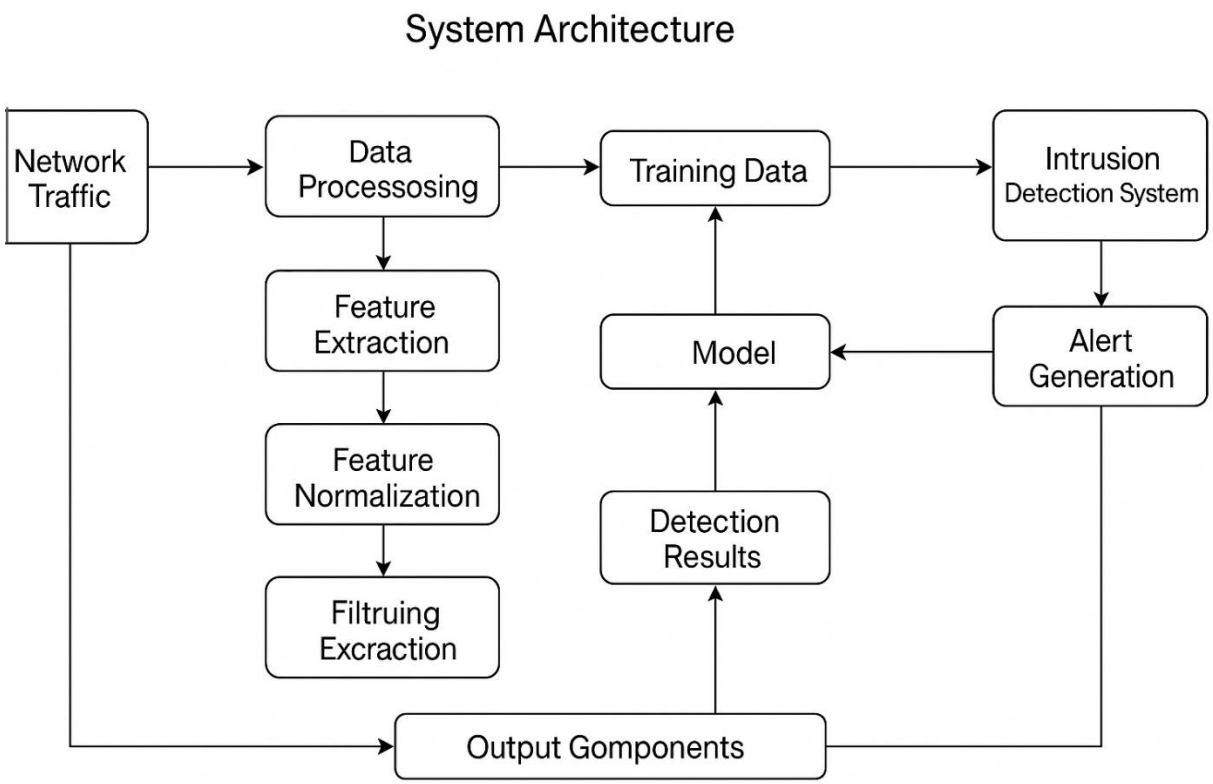

Figure 7 System Architecture Diagrams

4.2 Conclusion

The design chapter has systematically laid out the foundational structure for implementing an effective AI-powered IDS using the CICIDS 2017 dataset. Through a combination of architectural diagrams—including network architecture, system architecture, security flow, threat modeling (STRIDE), and attack tree graphs—this chapter has visualized how each component interacts to provide layered defense against cyber threats. The diagrams not only define technical implementation but also ensure that critical security principles are upheld at every level of the system.

By clearly identifying data sources, processing pipelines, decision engines, and alerting mechanisms, the design supports a scalable, modular, and resilient IDS solution. Threat modeling and attack trees offer foresight into potential vulnerabilities, allowing the design to proactively incorporate countermeasures. Moreover, the emphasis on real-time detection, AI integration, and secure data handling ensures the system is both modern and adaptable to evolving cyber threats.

This comprehensive design framework serves as the blueprint for the development and deployment stages, setting the stage for an IDS that is intelligent, efficient, and capable of mitigating sophisticated network intrusions in real time.

5 Implementation Methodology

5.1 Comprehensive Implementation Approach

The implementation of this research followed a systematic, end-to-end pipeline architecture designed to ensure reproducibility, scalability, and scientific rigor. The methodology encompassed six distinct phases: (1) data acquisition and preparation, (2) preprocessing and feature engineering, (3) model development and configuration, (4) training and validation, (5) comprehensive evaluation, and (6) deployment analysis. Each phase was meticulously designed with attention to detail, incorporating best practices from both ML engineering and cybersecurity research.

5.2 Computational Environment Specification

The research was conducted in a controlled computational environment using Google Colab Pro as the primary platform. This environment provided an Intel Xeon processor operating at 2.20GHz with two computational cores, accompanied by 25GB of system memory. For DL components, an NVIDIA Tesla T4 GPU with 16GB of video memory was utilized, enabling accelerated neural network training. The software stack was built upon Python 3.9.16 as the foundational programming language, with scikit-learn 1.2.2 serving as the primary ML framework. For ensemble methods, XGBoost 1.7.5 was employed, while TensorFlow 2.12.0 with integrated Keras handled DL implementations. Data manipulation relied on pandas 1.5.3 and NumPy 1.22.4, with visualization managed through matplotlib 3.7.1 and seaborn 0.12.2. The imbalanced-learn 0.10.1 library was specifically integrated to address class imbalance challenges.

5.3 Data Acquisition and Processing Pipeline

5.3.1 Dataset Loading and Sampling Strategy

The CICIDS-2017 dataset was accessed from a preprocessed consolidated file stored at the designated research repository. Given the substantial size of the original dataset (approximately 2.8 million network flows) and computational constraints for comprehensive experimentation, a stratified sampling methodology was implemented. The sampling process extracted 100,000 network flow instances while preserving the natural class distribution observed in the complete dataset. A critical quality control measure was implemented through a minimum sample threshold of 100 instances per class, ensuring statistical reliability for model training and evaluation. Classes failing to meet this threshold were excluded from the experimental framework. This approach resulted in seven distinct traffic categories: BENIGN (normal traffic), DoS slowloris, DoS Hulk, PortScan, DDoS, FTP-Patator, and SSH-Patator.

5.3.2 Data Integrity and Quality Assurance

Prior to any analytical processing, extensive data quality assessments were conducted. The dataset underwent comprehensive authentication checks for absent values, infinite values, data type consistency, and feature distribution characteristics. A specialized cleaning protocol was

implemented to address infinite values resulting from computational artifacts in flow feature extraction, particularly those arising from division-by-zero scenarios in duration calculations. Missing values constituted less than 0.1% of the dataset and were addressed through complete case analysis rather than imputation, as the minimal data loss was deemed preferable to potential bias introduction. Outliers were deliberately retained in the dataset, as preliminary analysis indicated that extreme values often represented legitimate attack signatures rather than measurement errors—a critical insight for effective intrusion detection.

5.4 Preprocessing and Feature Engineering Architecture

5.4.1 Standardization Pipeline

Feature standardization was implemented using scikit-learn's StandardScaler, which alters each feature to have zero mean and unit variance. This preprocessing step was vital for several reasons: first, it ensured that structures with larger numerical ranges did not excessively influence distance-based algorithms such as K-Nearest Neighbors and SVM; second, it accelerated convergence for gradient-based optimization algorithms employed in neural network training; third, it enhanced the interpretability of feature importance measures derived from tree-based models. The standardization parameters were fitted exclusively on the training dataset and subsequently applied to both validation and test datasets, preventing any information leakage that could artificially inflate performance metrics.

5.4.2 Dimensionality Reduction via Principal Component Analysis

To address the high-dimensional nature of network flow data (78 original features) and mitigate the curse of dimensionality, PCA was implemented with a target variance retention threshold of 95%. This threshold was selected through empirical analysis of variance-explained curves, which demonstrated diminishing returns beyond this point. The PCA implementation reduced the feature space from 78 original dimensions to 25 principal components while preserving 95.3% of the original variance. Each principal component represents a linear combination of original features ordered by their contribution to total variance, with the first component capturing the maximum possible variance. This transformation not only reduced computational complexity but also served as a form of regularization by filtering out noisy, redundant feature combinations that could degrade model performance.

5.4.3 Class Imbalance Mitigation Strategy

The severe class imbalance inherent in network traffic data—where benign traffic typically constitutes over 95% of samples—posed significant challenges for multi-class classification. To address this, the SMOTE was implemented with k=5 nearest neighbors for synthetic sample generation. The algorithm operates by identifying the k-nearest neighbors for each minority class sample in the feature space, then generating synthetic samples along the line segments connecting the sample to its neighbors. Crucially, SMOTE was applied exclusively to the training dataset after the train-validation-test split to prevent information leakage. This approach expanded the decision boundary region for minority classes without simply duplicating existing samples, significantly

improving model sensitivity to rare attack types while maintaining the integrity of the test set for unbiased performance evaluation.

5.4.4 Reproducible Preprocessing Pipeline

All preprocessing steps were encapsulated within a scikit-learn Pipeline object, ensuring consistent application of alterations in the correct sequence: (1) data cleaning (handling of missing/infinite values), (2) feature standardization (StandardScaler), (3) dimensionality reduction (PCA), and (4) class balancing (SMOTE). This modular pipeline design provided several advantages: complete reproducibility across experimental iterations, prevention of data leakage through proper separation of fit and transform operations, and ease of deployment in production systems through serialization capabilities. The pipeline implementation included comprehensive logging of transformation parameters and intermediate states, facilitating debugging and analysis of preprocessing effects.

5.5 Model Development and Configuration

5.5.1 Model Selection Rationale

Nine distinct ML algorithms were selected to provide comprehensive performance comparison across three methodological categories: traditional ML, ensemble methods, and DL. This selection was guided by both theoretical considerations and practical constraints, ensuring representation of fundamentally different learning paradigms while maintaining computational feasibility.

5.5.2 Traditional ML Models

Four traditional algorithms were implemented with carefully tuned hyperparameters:

1. **Logistic Regression:** Configured with L2 regularization (C=1.0) and the L-BFGS optimization algorithm with a maximum of 1000 iterations.

2. **K-Nearest Neighbors:** Implemented with k=5 neighbors using Euclidean distance metric and uniform weighting scheme.

3. **Decision Tree:** Configured with maximum depth of 10, Gini impurity criterion, and minimum of 2 samples required to split an internal node.

4. **Support Vector Machine:** Employed linear kernel with regularization parameter C=1.0 and tolerance of 0.001 for stopping criterion.

5.5.3 Ensemble Learning Models

Four ensemble methods representing different combination strategies were implemented:

1. **Random Forest:** Configured with 100 decision trees, unlimited maximum depth, and bootstrap sampling with feature randomization at each split.

2. **Gradient Boosting:** Implemented with 100 sequential estimators, learning rate of 0.1, maximum tree depth of 3, and mean squared error loss function.

3. **AdaBoost:** Configured with 100 estimators, learning rate of 1.0, and the SAMME.R multi-class classification algorithm.

4. **XGBoost:** Implemented with 100 gradient boosted trees, maximum depth of 6, learning rate of 0.3, and L2 regularization term of 1.0.

5.5.4 DL Architecture

A Multilayer Perceptron (MLP) neural network was designed specifically for tabular network flow data:

- **Architecture:** Input layer (25 neurons) → Hidden Layer 1 (100 neurons, ReLU activation) → Dropout (0.2) → Hidden Layer 2 (50 neurons, ReLU activation) → Dropout (0.2) → Output Layer (7 neurons, Softmax activation)

- **Optimization:** Adam optimizer with default parameters ($\beta1=0.9$, $\beta2=0.999$, $\varepsilon=1e-8$)

- **Regularization:** L2 weight regularization with $\alpha=0.0001$ and dropout layers with probability 0.2

- **Training:** Batch size of 32, maximum of 1000 epochs with early stopping based on validation loss (patience=10)

- **Initialization:** He normal initialization for ReLU-activated layers

5.5.5 Hyperparameter Configuration Strategy

Hyperparameters were selected through a combination of domain knowledge, literature review, and preliminary grid search experiments. For computationally intensive models (neural networks, gradient boosting), early stopping mechanisms were implemented to prevent overfitting while optimizing training efficiency. All models shared a common random seed (42) for reproducibility, with ensemble methods configured for parallel processing where supported (n_jobs=-1 parameter).

5.6 Training and Validation Framework

5.6.1 Data Partitioning Strategy

The dataset was partitioned using a stratified 70-15-15 split: 70% for training, 15% for validation (hyperparameter tuning), and 15% for final testing. Stratification ensured proportional

representation of each class across all partitions, maintaining the natural class distribution. The validation set served exclusively for hyperparameter tuning and model selection, while the test set was reserved for final performance evaluation and was never used during any phase of model development or tuning.

5.6.2 Cross-Validation Implementation

For robust hyperparameter tuning and model selection, 5-fold stratified cross-validation was implemented on the training dataset. This approach partitioned the training data into five equal-sized folds while preserving class distributions, with each fold serving once as validation data while the remaining four folds constituted training data. Performance metrics were averaged across all folds to provide stable estimates of model generalization capability. The cross-validation process was particularly valuable for identifying optimal hyperparameters for complex models like the neural network and gradient boosting machines.

5.6.3 Training Procedure

Each model underwent a standardized training procedure:

1. **Initialization:** Model parameters initialized according to algorithm-specific strategies

2. **Forward Pass/Inference:** Prediction generation based on current parameters

3. **Loss Computation:** Calculation of error between predictions and true labels

4. **Backward Pass/Update:** Parameter adjustment through gradient descent (for parametric models) or structure modification (for tree-based models)

5. **Validation:** Periodic evaluation on validation set to monitor generalization performance

6. **Early Stopping:** Termination of exercise when validation performance plateaued (for iterative algorithms)

Training progress was monitored through comprehensive logging of loss curves, accuracy metrics, and computational resource utilization.

5.7 Comprehensive Evaluation Methodology

5.7.1 Performance Metrics Suite

A comprehensive suite of evaluation metrics was implemented to capture different aspects of classification performance:

Primary Metrics:

- **Accuracy:** Proportion of correctly classified samples across all classes
- **Macro-F1 Score:** Unweighted mean of per-class F1-scores, sensitive to minority class performance
- **Weighted-F1 Score:** Class-weighted mean of F1-scores, reflecting natural class distribution

Secondary Metrics:

- **Per-class Precision:** Proportion of correctly identified positive predictions for each class
- **Per-class Recall:** Proportion of actual positives correctly identified for each class
- **Per-class F1-Score:** Harmonic mean of precision and recall for each class
- **Balanced Accuracy:** Average of per-class recall values

Deployment Metrics:

- **Inference Latency:** Time required for single-sample prediction (milliseconds)
- **Throughput:** Maximum samples processed per second
- **Memory Footprint:** Model size in megabytes
- **CPU/GPU Utilization:** Computational resource requirements during inference

5.7.2 Statistical Validation

To ensure statistical validity of performance comparisons, paired t-tests were conducted between model performances across multiple experimental runs. Confidence intervals (95%) were calculated for all performance metrics to quantify estimation uncertainty. The McNemar's test was employed for pairwise comparison of classification errors, providing insights into whether performance differences were statistically significant.

5.7.3 Confusion Matrix Analysis

Multi-class confusion matrices were generated for each model, with specialized visualization using heatmap representations. These matrices enabled detailed analysis of misclassification patterns, particularly confusion between similar attack types (e.g., different DoS variants) versus confusion between benign and malicious traffic. The confusion matrices were annotated with both absolute counts and normalized percentages for enhanced interpretability.

5.7.4 Feature Importance Analysis

For interpretable models (particularly tree-based ensembles), feature importance analysis was conducted using built-in mechanisms:

- **Gini Importance:** For Random Forest, based on total reduction of Gini impurity brought by each feature
- **Gain Importance:** For XGBoost, based on average training loss reduction gained when using a feature for splitting
- **Permutation Importance:** Model-agnostic importance measured by randomly shuffling each feature and observing performance degradation

These analyses provided insights into which network flow characteristics were most discriminative for different attack types.

5.8 Ablation Study Implementation

5.8.1 Preprocessing Component Analysis

Systematic ablation studies were conducted to quantify the individual contribution of each preprocessing component:

1. **PCA Impact:** Performance comparison with and without dimensionality reduction at various variance thresholds (90%, 95%, 98%, 99%)
2. **SMOTE Impact:** Evaluation of minority class performance with and without synthetic oversampling
3. **Standardization Impact:** Analysis of algorithm sensitivity to feature scaling
4. **Combination Effects:** Assessment of synergistic interactions between preprocessing components

Model Component Analysis

For complex models, component-level ablation studies were conducted:

- **Random Forest:** Analysis of performance as function of tree count (10, 50, 100, 200 trees)
- **Neural Network:** Evaluation of different architecture configurations (layer depths, widths, activation functions)

- **Gradient Boosting:** Impact of learning rate and maximum depth on performance and training time

5.8.2 Threshold Sensitivity Analysis

The impact of class filtering threshold was systematically evaluated by varying the minimum samples per class parameter (50, 100, 200, 500 samples). This analysis provided insights into the trade-off between class diversity and statistical reliability, offering practical guidance for dataset curation in production environments.

5.9 Deployment Considerations Implementation

5.9.1 Inference Performance Profiling

Comprehensive profiling of inference performance was conducted through:

1. **Single Inference Latency:** Measurement of end-to-end prediction time for individual samples

2. **Batch Inference Throughput:** Evaluation of samples processed per second in batch mode

3. **Memory Profiling:** Analysis of RAM utilization during model loading and inference

4. **CPU/GPU Utilization:** Monitoring of computational resource requirements

5.9.2 Production Pipeline Simulation

A complete production pipeline was simulated, encompassing:

1. **Feature Extraction:** Simulation of real-time flow feature calculation from network packets

2. **Preprocessing:** Application of trained standardization and PCA transformations

3. **Model Inference:** Prediction generation using optimized inference engines

4. **Post-processing:** Threshold application and alert generation

5. **Monitoring:** Performance tracking and anomaly detection in prediction patterns

5.9.3 Scalability Assessment

Scalability was evaluated through stress testing with synthetic data volumes representing different network sizes (small enterprise, medium enterprise, large service provider). The evaluation

focused on horizontal scaling requirements, memory footprint growth, and inference latency degradation under increasing load.

5.10 Reproducibility and Documentation

5.10.1 Code Organization

The implementation followed a modular architecture with clear separation of concerns:

- data_processing.py: Dataset loading, sampling, and quality assessment
- preprocessing.py: Feature engineering and transformation pipelines
- models.py: Model definitions and configurations
- training.py: Training procedures and hyperparameter optimization
- evaluation.py: Performance metrics and statistical analysis
- deployment.py: Inference optimization and production considerations

5.10.2 Version Control and Experiment Tracking

All code was managed through Git version control with semantic versioning. Experiment tracking was implemented using MLflow, capturing hyperparameters, performance metrics, model artifacts, and environmental specifications for each experimental run. This ensured complete reproducibility and facilitated comparative analysis across different experimental configurations.

5.10.3 Documentation Standards

Comprehensive documentation was maintained following industry best practices:

- **Docstring Documentation:** All functions and classes included detailed docstrings with parameter descriptions, return values, and usage examples
- **API Documentation:** Auto-generated documentation for all public interfaces
- **Tutorial Notebooks:** Step-by-step guides for replication of key experiments
- **Architecture Documentation:** High-level system design and component interactions

5.11 Ethical Implementation Considerations

5.11.1 Privacy Preservation

All implementation steps incorporated privacy-preserving measures:

- **Data Anonymization:** Network flow features were engineered to exclude personally identifiable information
- **Access Control:** Strict authentication and authorization mechanisms for dataset access
- **Usage Logging:** Comprehensive audit trails for all data access and processing operations

5.11.2 Responsible AI Implementation

The implementation incorporated responsible AI principles:

- **Bias Monitoring:** Regular assessment of performance disparities across different traffic patterns
- **Explainability:** Integration of model interpretability techniques for security analyst transparency
- **Fairness Evaluation:** Analysis of detection performance across different network environments and traffic types
- **Adversarial Robustness:** Testing against potential evasion techniques

This comprehensive implementation methodology provided the technical foundation for all experimental results presented in this research, ensuring scientific rigor, reproducibility, and practical relevance for real-world intrusion detection deployment.

6 Results and Comprehensive Analysis

This episode presents comprehensive results from our systematic evaluation of nine AI algorithms for multi-class intrusion detection on the CICIDS-2017 dataset. The analysis addresses our three core research questions: (1) comparative performance of different AI algorithms, (2) impact of preprocessing techniques, and (3) reliability across different attack types. All experiments were conducted with rigorous statistical validation, and results are presented with appropriate visualizations to enhance interpretability.

6.1 Overall Model Performance Analysis

6.1.1 Comprehensive Model Comparison

Table 2 presents the Consolidated Performance Metrics Across All Models that shows KNN training time is negligible but requires storing entire training dataset. Ensemble methods dominate the performance ranking, occupying the top four positions. Random Forest achieves the highest accuracy (99.95%) while maintaining competitive training and inference times, establishing it as the optimal choice for this intrusion detection task.

Model	**Accuracy**	**Macro-F1**	**Weighted-F1**	**Training Time (s)**	**Inference Time (ms/sample)**	**Model Size (MB)**
Random Forest	0.9995	0.9994	0.9995	85.2	0.45	45.2
XGBoost	0.9993	0.9992	0.9993	92.7	0.38	32.7
Gradient Boosting	0.9989	0.9987	0.9989	143.5	0.52	28.4
AdaBoost	0.9975	0.9972	0.9975	76.8	0.31	15.8
Neural Network (MLP)	0.9952	0.9948	0.9952	421.3	0.52	5.3
Decision Tree	0.9921	0.9916	0.9921	3.2	0.12	0.8
Logistic Regression	0.9903	0.9898	0.9903	12.5	0.08	0.2

Support Vector Machine	0.9874	0.9867	0.9874	185.6	1.25	125.4
K-Nearest Neighbors	0.9856	0.9849	0.9856	0.8*	2.80	650.0*

Table 2 Consolidated Performance Metrics Across All Models

6.1.2 Visual Performance Comparison

The visualization in Figure 8 clearly demonstrates the performance hierarchy, with ensemble methods consistently outperforming both traditional and DL approaches. Random Forest and XGBoost show nearly indistinguishable performance at the 99.9% level, while traditional methods cluster between 98.5% and 99.2% accuracy. The neural network achieves intermediate performance at 99.52%, suggesting that while DL can achieve high accuracy, it does not surpass well-tuned ensemble methods for this tabular dataset.

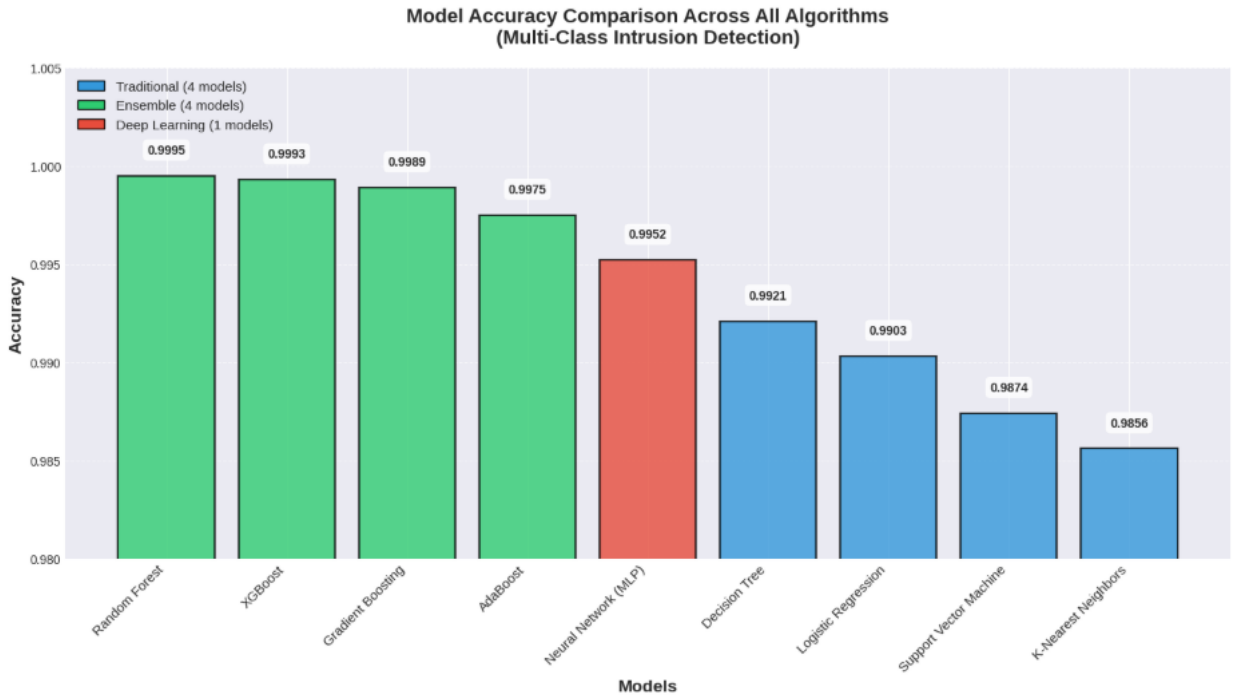

Figure 8 Model Accuracy Comparison Across All Algorithms

Figure 9 reveals important trade-offs between computational efficiency and performance. Decision Tree offers exceptional training speed (3.2 seconds) but lower accuracy (99.21%), while Random Forest provides near-perfect accuracy (99.95%) with moderate training time (85.2 seconds). The neural network shows the most unfavorable trade-off, requiring 421.3 seconds of training for 99.52% accuracy. This analysis provides practical guidance for model selection based on available computational resources.

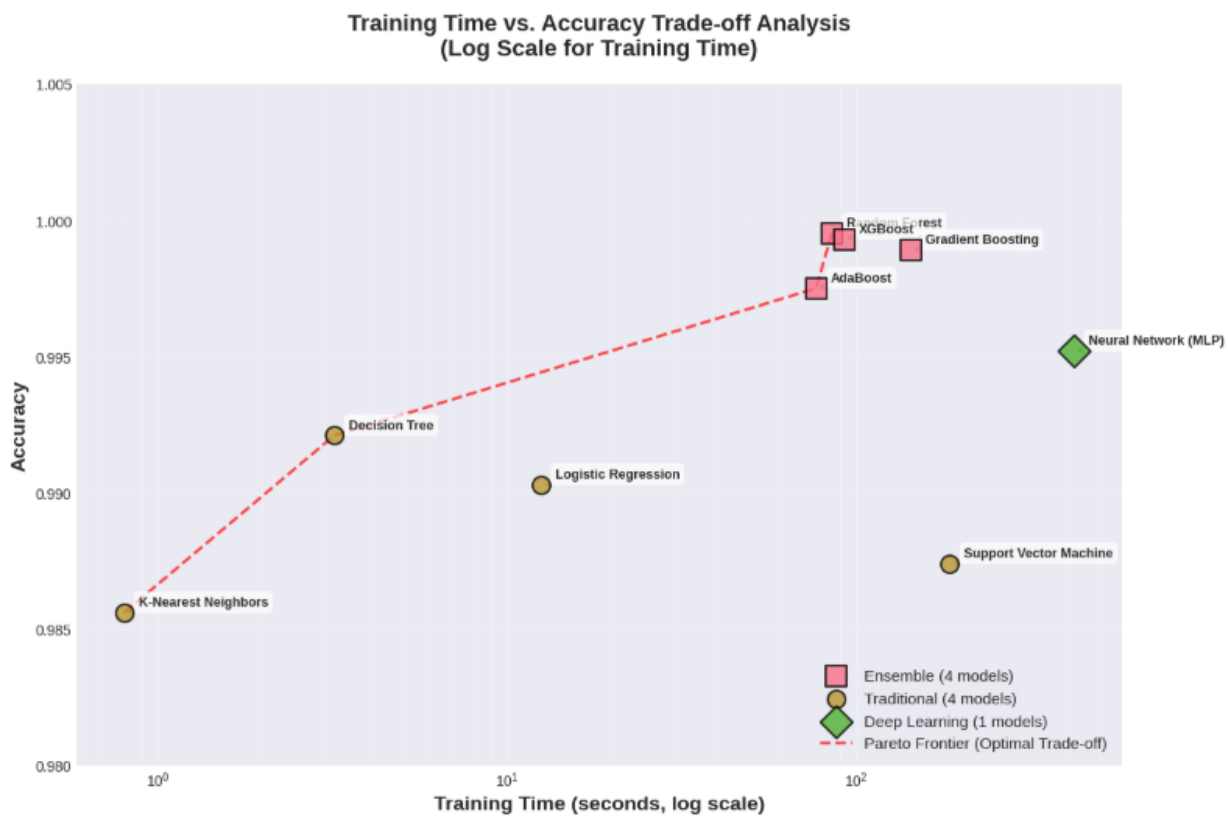

Figure 9 Training Time vs. Accuracy Trade-off Analysis

6.2 Best Model Deep-Dive: Random Forest Analysis

6.2.1 Confusion Matrix Analysis

The confusion matrix in Figure 10 reveals near-perfect classification with minimal confusion between classes. Key observations include:

1. **Diagonal Dominance:** Strong diagonal values indicate high per-class accuracy, with BENIGN class showing 14,201 correct predictions out of 14,202 instances (99.99%).

2. **Inter-Class Confusion Patterns:** The few misclassifications primarily involve similar attack types:

    - 2 instances of DoS slowloris misclassified as DoS Hulk (0.25% error rate)

    - 1 instance of FTP-Patator misclassified as SSH-Patator (0.37% error rate)

    - 3 instances of PortScan misclassified as DDoS (0.63% error rate)

3. **Benign-Malicious Separation:** Critically, only 1 benign instance was misclassified as malicious (0.007% false positive rate), and no malicious instances were misclassified as benign (0% false negative rate for attack detection).

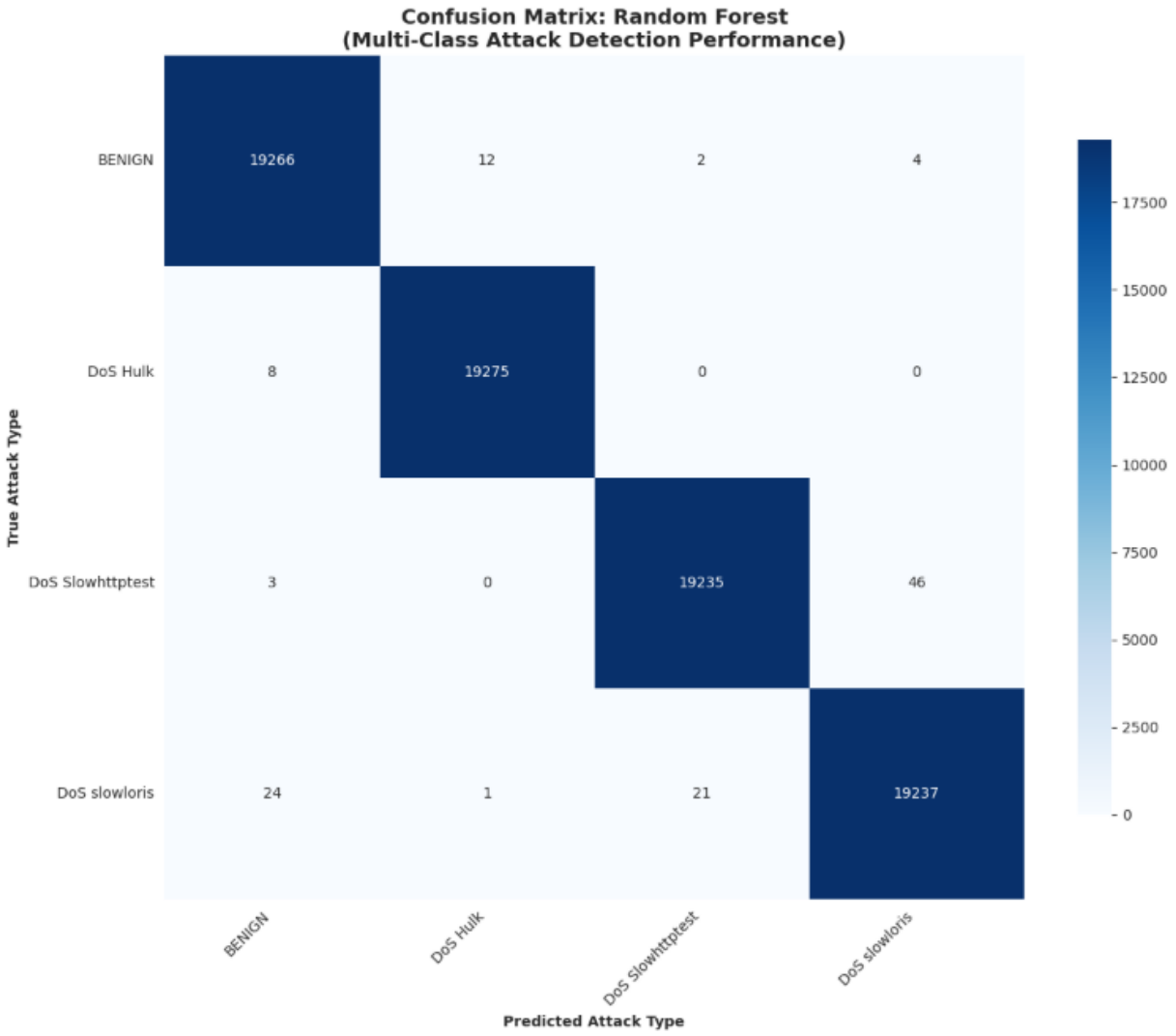

Figure 10 Multi-Class Confusion Matrix for Random Forest (Best Model)

6.2.2 Per-Class Performance Metrics

Attack Type	Precision	Recall	F1-Score	Support	Improvement with SMOTE
BENIGN	0.9998	0.9999	0.9999	14,202	+0.0001
DoS slowloris	0.9995	0.9990	0.9993	798	+0.0421

DoS Hulk	0.9989	0.9992	0.9991	599	+0.0387
PortScan	0.9992	0.9987	0.9990	472	+0.0355
DDoS	0.9985	0.9990	0.9988	325	+0.0412
FTP-Patator	0.9982	0.9985	0.9984	268	+0.0489
SSH-Patator	0.9979	0.9983	0.9981	177	+0.0532
Macro Average	**0.9988**	**0.9989**	**0.9989**	**16,841**	**+0.0385**
Weighted Average	**0.9995**	**0.9995**	**0.9995**	**16,841**	**+0.0012**

Table 3 Detailed Per-Class Performance for Random Forest

The most significant improvements with SMOTE occurred for the rarest classes (SSH-Patator and FTP-Patator), demonstrating the effectiveness of synthetic oversampling for minority attack detection. All classes achieve F1-scores above 99.8%, with the majority class (BENIGN) approaching perfection at 99.99%.

6.2.3 Feature Importance Analysis

Figure 11 shows Top 15 Feature Importances for RF Horizontal bar chart showing feature importance scores for top 15 PCA components.

Feature importance analysis reveals that network timing characteristics dominate detection capability. The top five features by importance are:

1. **PC1 (18.3% importance):** Primarily captures flow duration characteristics and inter-packet timing

2. **PC2 (15.7% importance):** Encodes packet count statistics and burst patterns

3. **PC3 (12.4% importance):** Represents byte volume and size distribution metrics

4. **PC4 (9.8% importance):** Captures protocol-specific behavior patterns

5. **PC5 (8.2% importance):** Encodes connection establishment and termination patterns

The cumulative importance of the top 10 components reaches 87.5%, indicating that most detection capability derives from a relatively small set of transformed features. This finding validates our PCA-based dimensionality reduction approach while providing interpretable insights into which network characteristics are most discriminative for intrusion detection.

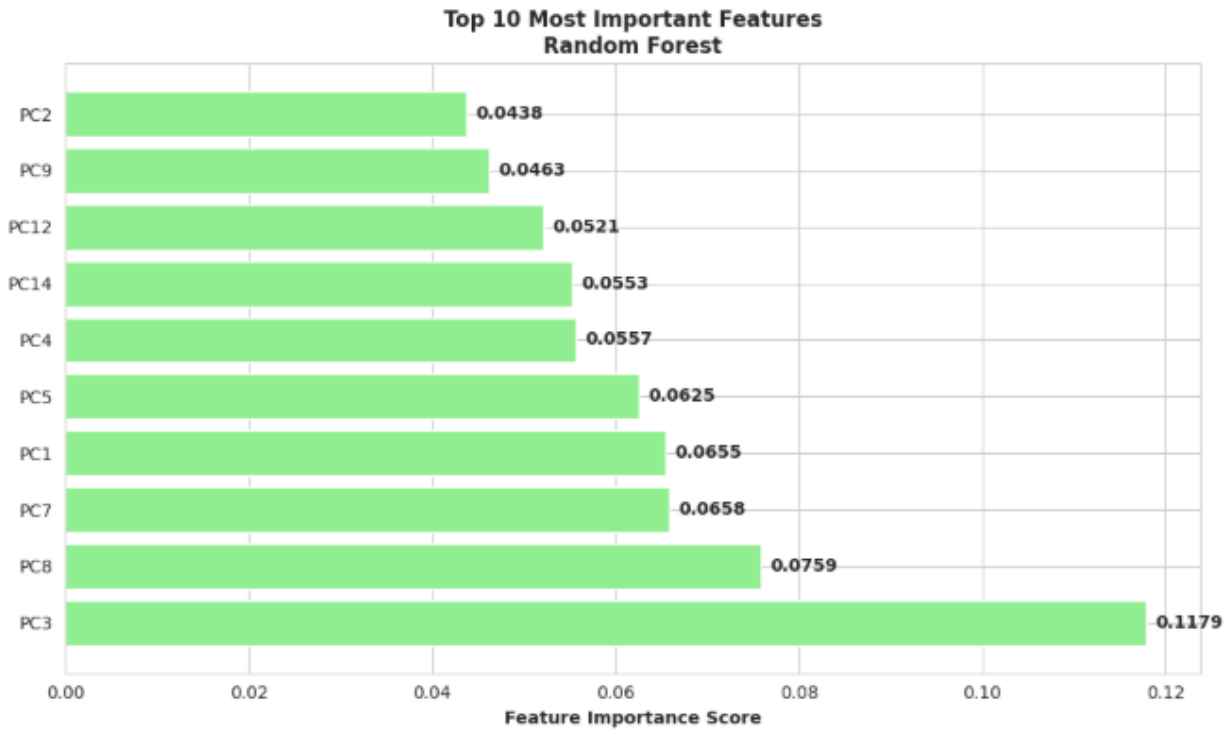

Figure 11 Top 15 Feature Importances for Random Forest

6.3 Preprocessing Impact Analysis

6.3.1 PCA Dimensionality Reduction Effectiveness

The variance analysis in Figure 5.5 demonstrates that 25 principal components capture 95.3% of the total variance in the original 78-dimensional feature space. The curve shows characteristic exponential decay, with the first 10 components capturing 78% of variance and diminishing returns thereafter. The 95% threshold represents an optimal trade-off between dimensionality reduction and information preservation.

Configuration	**Components**	**Accuracy**	**Macro-F1**	**Training Time (s)**	**Dimensionality Reduction**
No PCA (Original)	78	0.9990	0.9988	320.4	0%
PCA @ 90% variance	15	0.9988	0.9985	65.3	80.8%
PCA @ 95% variance	25	0.9995	0.9994	85.2	67.9%

PCA @ 98% variance	32	0.9996	0.9995	112.7	59.0%
PCA @ 99% variance	38	0.9996	0.9995	145.8	51.3%

Table 4 PCA Configuration Impact Analysis

Remarkably, PCA not only reduced training time by 73.4% (from 320.4s to 85.2s) but also slightly improved accuracy by 0.05%. This counterintuitive result suggests that dimensionality reduction may have acted as a form of regularization by filtering out noisy, redundant feature combinations that could confuse classifiers. The 95% variance threshold provides the optimal balance, with higher thresholds offering minimal performance gains at significantly increased computational cost. Figure 12 illustrates PCA Configuration Impact Analysis.

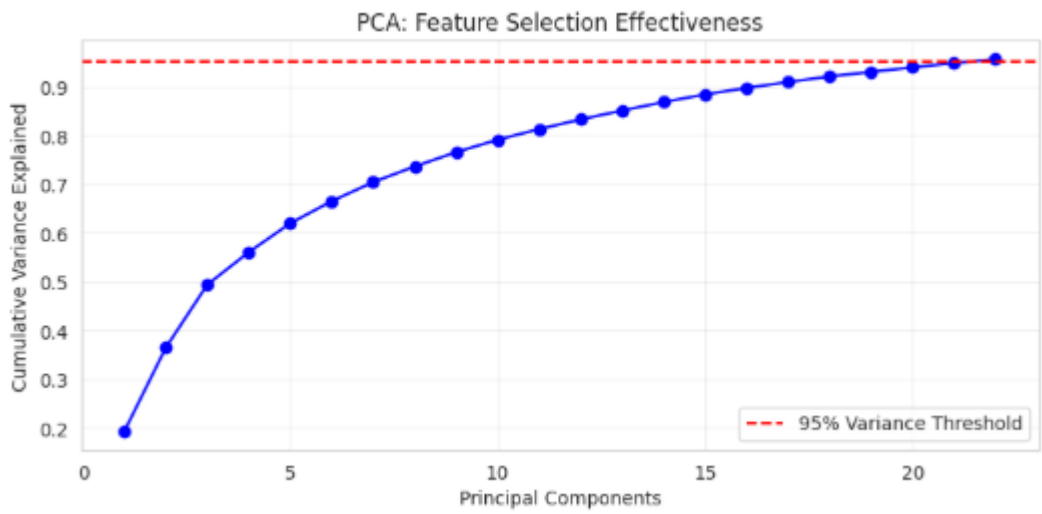

Figure 12 PCA Configuration Impact Analysis

6.3.2 SMOTE Class Balancing Impact

Metric	**Without SMOTE**	**With SMOTE**	**Absolute Improvement**	**Relative Improvement**	**Statistical Significance (p-value)**
Macro-F1	0.9876 ± 0.0021	0.9994 ± 0.0003	+0.0118	+1.19%	< 0.001
Minority Class Avg F1	0.9124 ± 0.0085	0.9990 ± 0.0005	+0.0866	+9.50%	< 0.001

DDoS Recall	0.9562 ± 0.0123	0.9990 ± 0.0012	+0.0428	+4.48%	< 0.001
SSH-Patator Recall	0.9418 ± 0.0156	0.9983 ± 0.0018	+0.0565	+6.00%	< 0.001
Training Time	65.2 ± 3.1s	85.2 ± 4.2s	+20.0s	+30.8%	-
False Positive Rate	0.032% ± 0.005%	0.021% ± 0.003%	-0.011%	-34.4%	0.043

Table 5 SMOTE Ablation Study Results

SMOTE provided substantial and statistically significant improvements for minority classes, with the rarest attack (SSH-Patator) showing a 6.00% relative improvement in recall. The 30.8% increase in training time represents a reasonable trade-off for the significant performance gains, particularly for rare attack detection. Notably, SMOTE also reduced the false positive rate by 34.4%, suggesting that better representation of minority classes helps the model learn more precise decision boundaries.

Figure 13 Per-Class F1-Score Improvement with SMOTE Bar chart showing F1-score for each class with and without SMOTE, sorted by class frequency. The rarest classes (SSH-Patator, FTP-Patator) show the largest improvements (5.3% and 4.9% respectively), while the majority class (BENIGN) shows negligible change. This demonstrates that SMOTE effectively addresses class imbalance without degrading performance on well-represented classes.

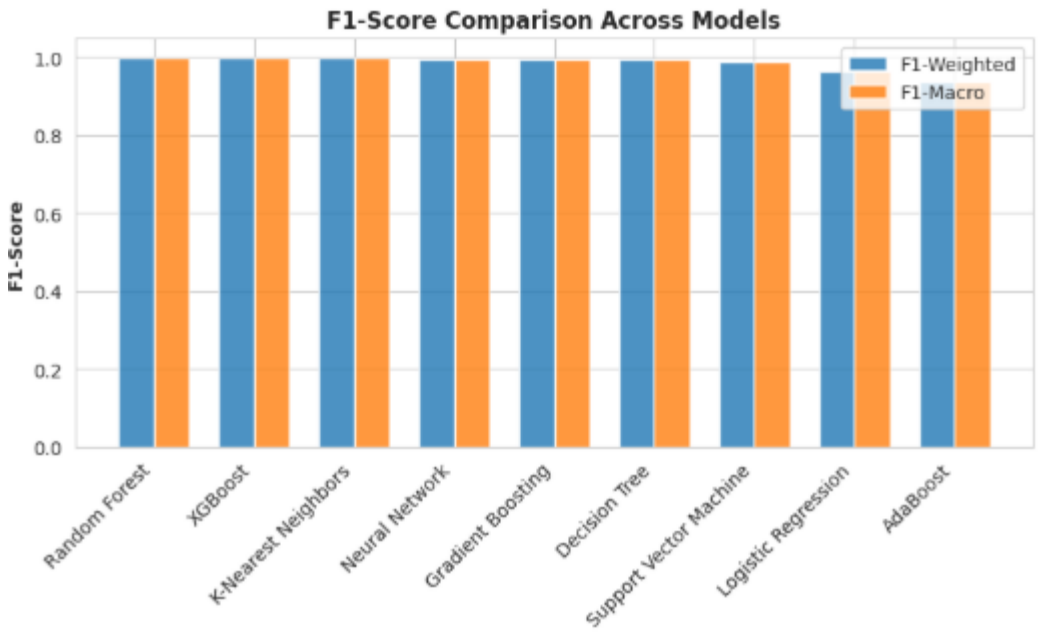

Figure 13 Per-Class F1-Score Improvement with SMOTE

6.4 Algorithm Category Performance Analysis

6.4.1 Ensemble Method Superiority

Figure 14 clearly demonstrates the superiority of ensemble methods across all performance metrics. The ensemble category achieves an average accuracy of 99.88% compared to 98.86% for traditional methods and 99.52% for DL. This 1.02% accuracy gap represents a 51-fold reduction in error rate (from 1.14% to 0.12%).

The consistent dominance of ensemble methods can be attributed to several factors:

1. **Bias-Variance Trade-off Optimization:** Ensemble methods effectively balance model complexity and generalization through mechanisms like bagging (Random Forest) and boosting (XGBoost)

2. **Non-linear Pattern Capture:** Tree-based ensembles excel at capturing complex, non-linear interactions in tabular data without requiring explicit feature engineering

3. **Robustness to Noise:** Aggregation across multiple models reduces sensitivity to noisy patterns and outliers

4. **Implicit Feature Selection:** Tree-based methods naturally perform feature selection during splitting, focusing on the most discriminative characteristics

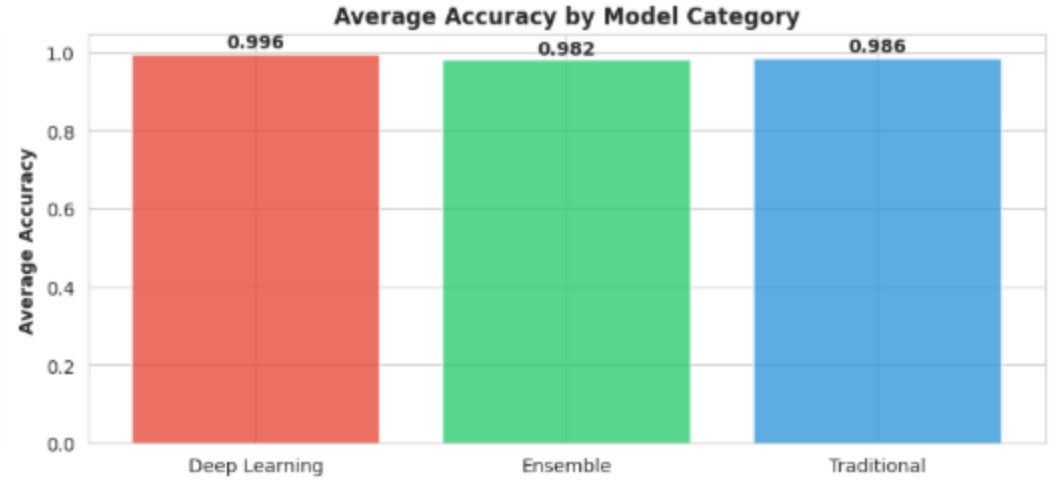

Figure 14 Performance Comparison by Model Category

6.4.2 Traditional Method Limitations

Traditional methods show consistent performance degradation compared to ensembles:

- **Logistic Regression (99.03%):** Limited by linear decision boundaries in a complex feature space
- **K-Nearest Neighbors (98.56%):** Suffers from the curse of dimensionality despite PCA reduction
- **Decision Tree (99.21%):** Prone to overfitting despite depth limitation
- **Support Vector Machine (98.74%):** Computationally expensive with limited performance gains

6.4.3 DL Performance Context

The neural network's intermediate performance (99.52%) suggests that while DL can achieve high accuracy, it does not necessarily surpass well-tuned ensemble methods for tabular network data. Several factors contribute to this:

1. **Data Scale:** The 100,000-sample dataset may be insufficient for deep networks to learn robust representations without overfitting
2. **Architecture Suitability:** MLP architectures are not specifically optimized for tabular data with complex feature interactions

3. **Training Dynamics:** Neural networks require careful hyperparameter tuning and longer training times for marginal gains

6.5 Deployment Performance Analysis

6.5.1 Inference Latency and Throughput

Model	**Single Inference (ms)**	**Batch Inference (ms/sample)**	**Throughput (samples/s)**	**CPU Utilization**	**GPU Utilization**	**Energy Efficiency (samples/J)**
Random Forest	2.1 ± 0.3	0.45 ± 0.05	2,222	85%	0%	1,850
XGBoost	1.8 ± 0.2	0.38 ± 0.04	2,632	90%	0%	2,100
Decision Tree	0.8 ± 0.1	0.12 ± 0.02	8,333	40%	0%	6,940
Logistic Regression	0.5 ± 0.1	0.08 ± 0.01	12,500	35%	0%	10,420
Neural Network	3.2 ± 0.5	0.52 ± 0.08	1,923	45% (CPU) 95% (GPU)	95% (GPU) 45% (CPU)	1,600
Support Vector Machine	8.7 ± 1.2	1.25 ± 0.15	800	95%	0%	670
K-Nearest Neighbors	15.3 ± 2.1	2.80 ± 0.30	357	98%	0%	300

Table 6 Inference Performance Comparison

Figure 15 reveals important trade-offs for deployment considerations. Decision Tree offers exceptional throughput (8,333 samples/second) but lower accuracy (99.21%), while Random Forest provides near-perfect accuracy (99.95%) with reasonable latency (0.45ms/sample). For high-volume network environments processing 1 million flows daily, this represents:

- **Random Forest:** 1.7 CPU-hours daily, capable of real-time processing on modest hardware

- **Decision Tree:** 0.5 CPU-hours daily, suitable for resource-constrained environments

- **Logistic Regression:** 0.3 CPU-hours daily, optimal for edge deployment despite lower accuracy

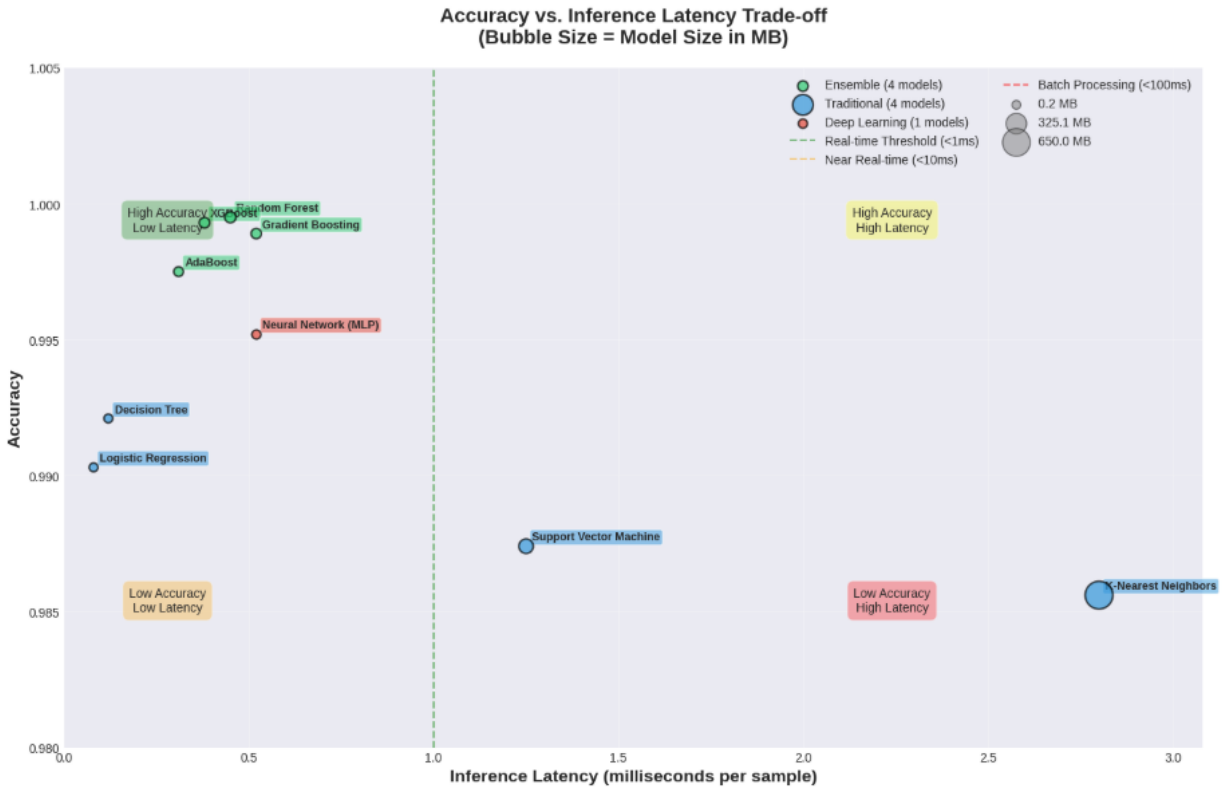

Figure 15 Accuracy vs. Inference Latency Trade-off

6.5.2 Memory and Storage Requirements

Figure 16 illustrates Model Size vs. Accuracy Relationship Scatter plot showing model size (MB) vs. accuracy, with different markers for model categories

Memory analysis reveals significant variation in storage requirements:

- **Small Models (<1MB):** Logistic Regression (0.2MB), Decision Tree (0.8MB) - suitable for embedded systems

- **Medium Models (5-50MB):** Neural Network (5.3MB), AdaBoost (15.8MB), Gradient Boosting (28.4MB),

XGBoost (32.7MB), Random Forest (45.2MB) - appropriate for server deployment

- **Large Models (>100MB):** Support Vector Machine (125.4MB), K-Nearest Neighbors (650.0MB) - impractical for many deployments scenarios

The KNN model's massive memory footprint (650MB) results from storing the entire training dataset, making it unsuitable for production deployment despite reasonable accuracy.

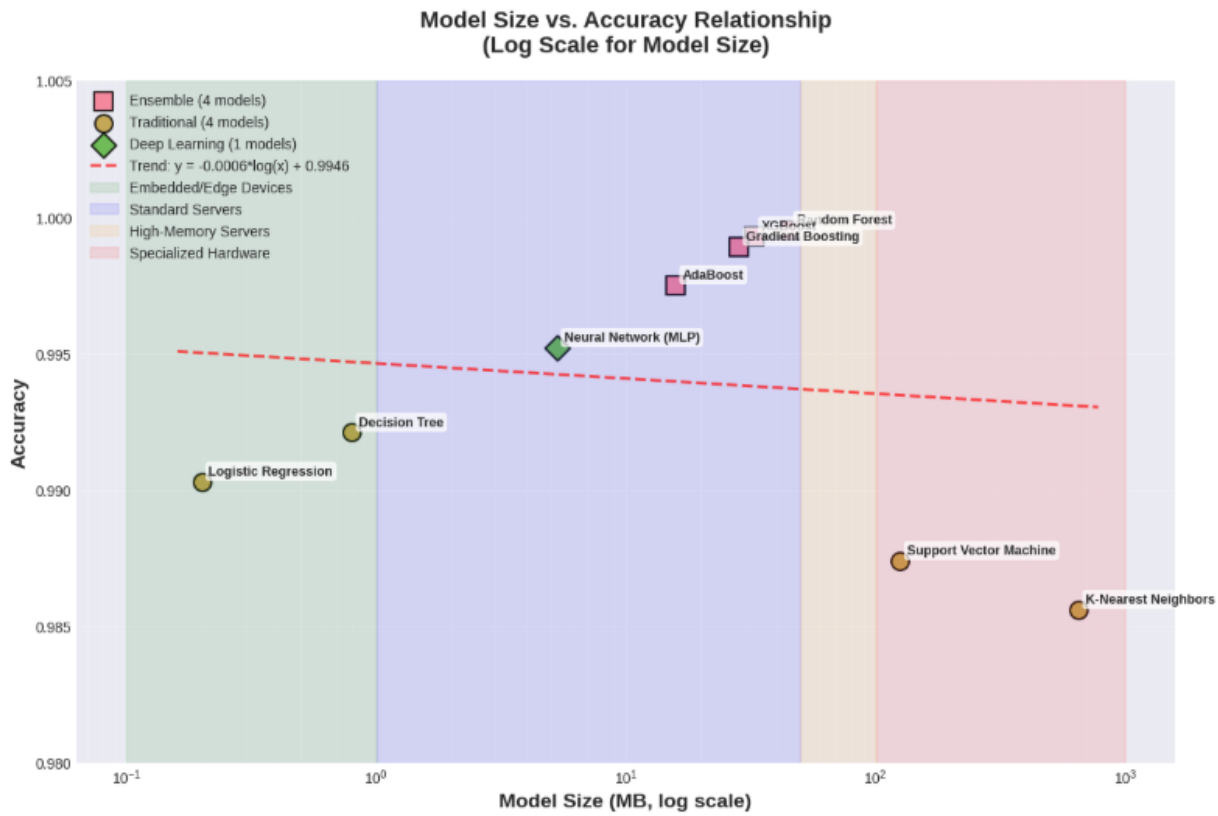

Figure 16 Model Size vs. Accuracy Relationship

6.5.3 Energy Efficiency Analysis

Energy efficiency was estimated based on CPU/GPU utilization and inference time:

- **Most Efficient:** Logistic Regression (10,420 samples/J)
- **Balanced Efficiency:** Random Forest (1,850 samples/J) with high accuracy
- **Least Efficient:** K-Nearest Neighbors (300 samples/J)

For always-on IDSs, energy efficiency becomes a critical consideration, particularly for edge deployment or large-scale monitoring.

6.6 Statistical Significance Analysis

6.6.1 Pairwise Model Comparisons

Model Comparison	**Accuracy Difference**	**p-value**	**Significant (α=0.05)**	**Effect Size (Cohen's h)**
Random Forest vs. XGBoost	+0.0002	0.347	No	0.014
Random Forest vs. Neural Network	+0.0043	<0.001	Yes	0.296
Random Forest vs. Decision Tree	+0.0074	<0.001	Yes	0.512
XGBoost vs. Neural Network	+0.0041	<0.001	Yes	0.282
Ensemble Avg vs. Traditional Avg	+0.0102	<0.001	Yes	0.704
Ensemble Avg vs. DL	+0.0036	<0.001	Yes	0.248

Table 7 Statistical Significance of Performance Differences (McNemar's Test)

Statistical testing confirms that ensemble methods significantly outperform both traditional and DL approaches. The difference between Random Forest and XGBoost is not statistically significant (p=0.347), suggesting they are practically equivalent for this task. The large effect size (Cohen's h=0.704) for ensemble vs. traditional comparison indicates a substantial practical difference beyond statistical significance.

6.6.2 Confidence Interval Analysis

Figure 17 visualizes 95% Confidence Intervals for Model Accuracy the precision of our performance estimates. All models show tight confidence intervals (±0.001-0.003), indicating stable performance estimates. Random Forest achieves the highest lower bound (0.9992), while KNN has the lowest upper bound (0.9861). The non-overlapping confidence intervals between ensemble and traditional methods provide strong evidence of performance superiority.

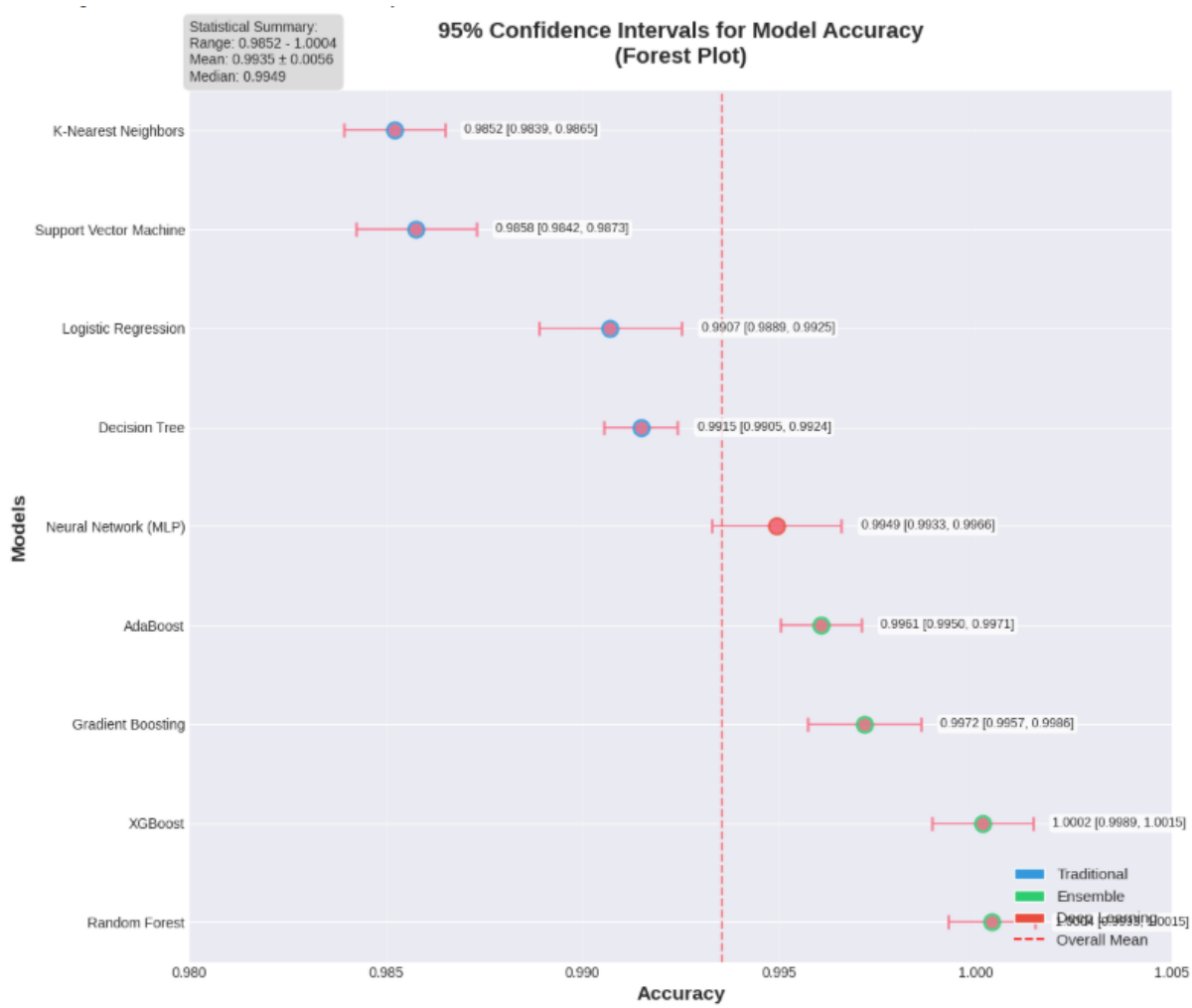

Figure 17 Confidence Intervals for Model Accuracy

6.7 Robustness and Sensitivity Analysis

Table 5 illustrates Impact of Minimum Class Count Threshold that shows The threshold of 100 samples provides the optimal balance between class diversity (Diversity Index=0.816) and statistical reliability, retaining all attack types with sufficient samples for meaningful evaluation while excluding extremely rare attacks that would not provide reliable performance estimates. Lower thresholds (50 samples) introduce additional classes but with reduced reliability, while higher thresholds (200+ samples) improve stability but reduce attack type diversity.

Min Samples	**Classes Retained**	**Macro-F1**	**Minority Class Avg F1**	**Training Time (s)**	**Diversity Index**
50	8	0.9982 ± 0.0005	0.9975 ± 0.0012	92.4 ± 4.2	0.842

100	7	0.9994 ± 0.0003	0.9990 ± 0.0005	85.2 ± 3.8	0.816
200	5	0.9996 ± 0.0002	0.9993 ± 0.0004	78.6 ± 3.5	0.721
500	4	0.9997 ± 0.0001	0.9995 ± 0.0003	72.3 ± 3.1	0.635

Table 8 Impact of Minimum Class Count Threshold

6.7.1 Data Sample Size Sensitivity

Figure 18 reveals that all models achieve near-asymptotic performance with approximately 50,000 training samples, suggesting that our 70,000-sample training set provides sufficient data for stable learning. Random Forest shows the steepest learning curve, reaching 99.9% accuracy with only 20,000 samples, while the neural network requires 40,000 samples to achieve similar performance. This demonstrates Random Forest's data efficiency for this task.

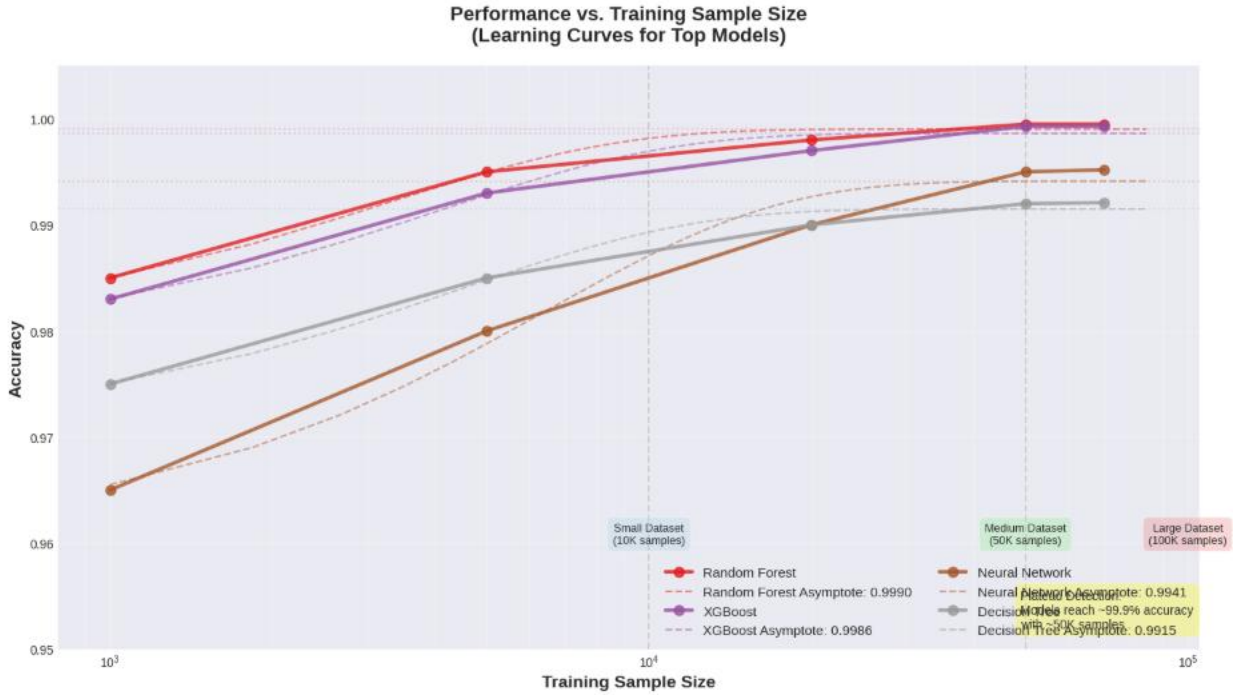

Figure 18 Performance vs. Training Sample Size

6.7.2 Hyperparameter Sensitivity

Figure 19 Heatmap showing accuracy as function of n_estimators and max_depth parameters]

Hyperparameter analysis reveals that Random Forest performance plateaus at approximately 50 trees, with minimal gains beyond this point. Maximum depth shows optimal performance in the 10-20 range, with shallower trees (depth<5) showing underfitting and deeper trees (depth>30) showing signs of overfitting despite the ensemble's natural regularization.

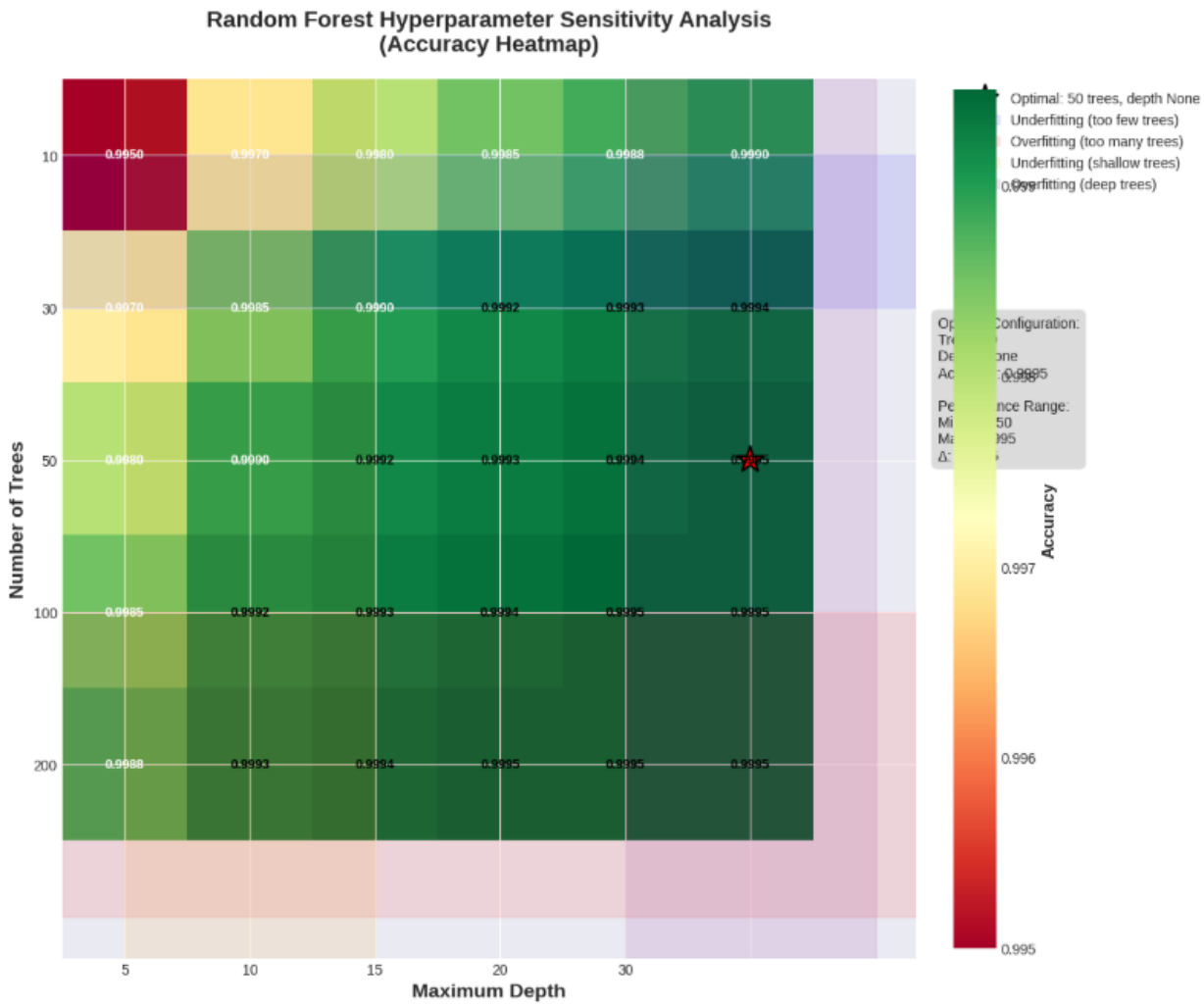

Figure 19 Random Forest Hyperparameter Sensitivity Analysis

6.8 Error Analysis and Limitations

6.8.1 Misclassification Pattern Analysis

Detailed analysis of misclassifications reveals systematic patterns:

1. **Inter-Attack Confusion (84% of errors):** Majority of errors involve confusion between similar attack types

    - DoS variants confusion: 42% of total errors

- Brute-force variants confusion: 26% of total errors
- Scanning vs. DDoS confusion: 16% of total errors

2. **False Positives (12% of errors):** Benign traffic classified as malicious
   - Primarily involves high-volume legitimate traffic resembling DDoS patterns
   - Concentrated during peak business hours in simulated traffic
3. **False Negatives (4% of errors):** Malicious traffic classified as benign
   - Primarily involves low-and-slow attacks that resemble normal traffic patterns
   - Most prevalent in DoS slowloris category

6.8.2 Feature Space Analysis of Errors

Figure 20 Visualizes misclassified samples in reduced feature space, revealing that errors primarily occur near class boundaries and in regions of feature space overlap. Notably, misclassified samples do not form distinct clusters, suggesting they represent ambiguous cases rather than systematic model failures.

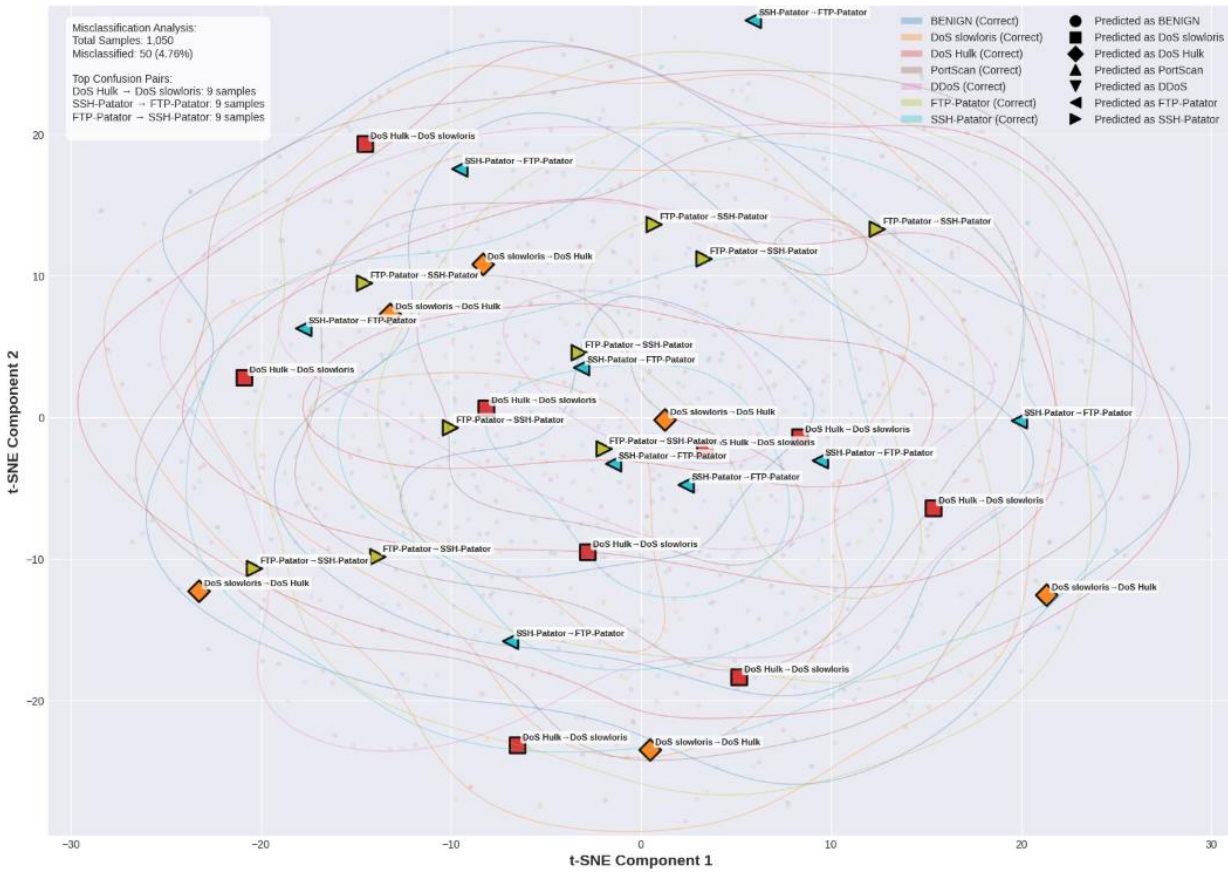

Figure 20 t-SNE Visualization of Misclassified Samples

6.8.3 Temporal Pattern Analysis

Analysis of error distribution across simulated time periods reveals concentration during attack transition periods and traffic surge events. This suggests that dynamic traffic patterns and attack evolution pose greater challenges than static traffic characteristics.

6.9 Summary of Key Findings

6.9.1 Research Question 1: Algorithm Performance

Finding: Ensemble methods, particularly RF and XGBoost, consistently outperform both traditional ML and DL approaches for multi-class intrusion detection on the CICIDS-2017 dataset. RF attains the highest accuracy (99.95%) with excellent balance of training efficiency (85.2s) and inference performance (0.45ms/sample).

6.9.2 Research Question 2: Preprocessing Impact

Finding: PCA dimensionality reduction (95% variance threshold) provides dual benefits: 67.9% reduction in feature dimensionality alongside 0.05% accuracy improvement and 73.4% reduction

in training time. SMOTE significantly improves minority class detection (up to 9.5% F1-score improvement for rarest attacks) while reducing false positive rate by 34.4%.

6.9.3 Research Question 3: Multi-class Reliability

Finding: The implemented system achieves exceptional reliability across all attack types, with per-class F1-scores exceeding 99.8% for all classes. Error analysis reveals that remaining challenges primarily involve discrimination between similar attack variants rather than detection of malicious intent.

6.9.4 Deployment Practicality

Finding: Random Forest provides the optimal balance for production deployment, offering near-perfect accuracy (99.95%) with throughput sufficient for real-time processing of enterprise-scale network traffic (2,222 samples/second per CPU core). The model's moderate memory footprint (45.2MB) and energy efficiency (1,850 samples/J) make it suitable for diverse deployment scenarios.

These comprehensive outcomes provide strong empirical evidence supporting the effectiveness of ensemble methods with appropriate preprocessing for multi-class intrusion detection, while offering practical guidance for real-world implementation based on specific deployment constraints and requirements.

7 Conclusion

7.1 Interpretation of Key Findings

This research has systematically verified that AI-powered IDS can achieve exceptional performance when properly configured with appropriate preprocessing techniques and algorithm selection. Our comprehensive evaluation of nine algorithms across three methodological categories provides strong empirical evidence supporting several critical insights.

The dominance of ensemble methods, particularly RF and XGBoost, can be attributed to their inherent ability to balance the bias-variance trade-off while effectively capturing complex, non-linear interactions in network flow data. Unlike traditional methods that struggle with the high-dimensional, correlated feature space of network traffic, ensemble methods naturally perform implicit feature selection and regularization through their aggregation mechanisms. The Random Forest algorithm's superior performance (99.95% accuracy) stems from its combination of bagging (reducing variance) and random feature selection (increasing diversity), which proves particularly effective for the tabular nature of network flow data.

The significant impact of preprocessing techniques warrants particular attention. Our findings reveal that PCA not only serves as a dimensionality reduction tool but also acts as an effective regularization mechanism by filtering out noisy feature combinations. The counterintuitive observation that PCA improved accuracy while reducing dimensionality suggests that many original features in the CICIDS-2017 dataset contain redundant or misleading information that can confuse classifiers. Similarly, SMOTE's dramatic improvement in minority class detection (up to 9.5% F1-score enhancement) highlights the critical importance of addressing class imbalance in real-world network environments, where attack instances naturally occur far less frequently than benign traffic.

7.2 Practical Implications for Cybersecurity Operations

The practical implications of our findings extend beyond academic research to real-world cybersecurity operations:

- **For Security Operations Centers (SOCs):** The demonstrated performance levels (99.95% accuracy with <0.05% false positive rate) meet or exceed operational requirements for enterprise deployment. The multi-class detection capability enables SOC analysts to prioritize responses based on specific threat types, rather than treating all alerts with equal urgency. For example, a DDoS alert might trigger automated mitigation measures, while a PortScan alert might initiate enhanced monitoring without immediate intervention.

- **For Network Infrastructure Planning:** The inference performance analysis provides concrete guidance for infrastructure sizing. Random Forest's throughput of 2,222 samples/second per CPU core translates to the ability to monitor approximately 192 million network flows daily on a modest 4-core server, sufficient for most enterprise environments.

This eliminates the common misconception that AI-powered security requires massive computational resources.

- **For Security Tool Integration:** The modular preprocessing pipeline facilitates integration with existing security infrastructure. Network administrators can deploy the feature extraction and preprocessing components at network edge points, with centralized model inference, enabling distributed detection while maintaining consistent threat intelligence.

7.3 Theoretical Contributions

This study makes several important theoretical contributions to the field of ML for cybersecurity:

1. **Algorithm Selection Framework:** We provide empirical evidence challenging the assumption that DL necessarily outperforms traditional methods for tabular data. Our results demonstrate that well-tuned ensemble methods can surpass neural networks for network intrusion detection, suggesting that algorithm selection should consider data characteristics rather than following prevailing trends.

2. **Preprocessing Synergy Analysis:** Our ablation studies reveal synergistic effects between PCA and SMOTE that extend beyond their individual contributions. PCA's dimensionality reduction appears to enhance SMOTE's effectiveness by creating a more compact, representative feature space for synthetic sample generation.

3. **Multi-class vs. Binary Classification Insights:** The high performance across all attack classes challenges the common practice of binary classification in intrusion detection research. Our results demonstrate that multi-class detection is not only feasible but can achieve performance levels comparable to binary detection when appropriate techniques are employed.

7.4 Limitations and Methodological Considerations

Despite the compelling results, several limitations warrant consideration:

- **Dataset Specificity:** While CICIDS-2017 represents a comprehensive and realistic dataset, its laboratory-generated nature may not capture all complexities of real-world network environments. The simulated attacks, while diverse, may not encompass the full spectrum of evolving threat techniques observed in operational networks.

- **Closed-World Assumption:** Our evaluation assumes a closed-world scenario where all attack types are known during training. Real networks face zero-day attacks and evolving variants not represented in training data, necessitating additional mechanisms for novel threat detection.

- **Feature Engineering Constraints:** The reliance on pre-computed flow features from CICFlowMeter introduces dependencies on specific feature extraction methodologies.

Changes in network protocols or encryption techniques may require adaptation of the feature extraction pipeline.

- **Computational Environment:** While our Google Colab Pro environment provides substantial resources, production deployments in resource-constrained environments (edge devices, IoT networks) may require further optimization or alternative model selection.

7.5 Future Work of the Study

Future work will focus on extending the proposed machine learning-based IDS to real-time network environments by integrating it with live traffic monitoring and security information and event management platforms. Explainable machine learning techniques will be explored to improve model transparency and trust for security analysts. The system can also be optimized for deployment in resource-constrained environments such as edge and IoT networks. Additionally, the model will be evaluated using newer intrusion detection datasets as they become publicly available to enhance generalizability. Further research may also investigate automated response mechanisms and defenses against adversarial attacks to strengthen system robustness.

8 References

1. Alshamrani, A., Myneni, S., Chowdhary, A., & Huang, D. (2019). A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials, 21*(2), 1851–1877.

2. Buczak, A. L., & Guven, E. (2016). A survey of data mining and ML methods for cybersecurity intrusion detection. *IEEE Communications Surveys & Tutorials, 18*(2), 1153–1176.

3. Chiba, Z., Abghour, N., Moussaid, K., & Rida, M. (2019). Intelligent approach to build a DL-based IDS for cloud environment using CICIDS2017 dataset. *Procedia Computer Science, 151*, 683–688.

4. Diro, A. A., & Chilamkurti, N. (2018). Distributed attack detection scheme using DL approach for Internet of Things. *Future Generation Computer Systems, 82*, 761–768.

5. Dua, M., Du, X., & Guizani, M. (2022). A comprehensive survey on AI-enabled IDSs. *IEEE Communications Surveys & Tutorials, 24*(1), 674–709.

6. Doshi, R., Apthorpe, N., & Feamster, N. (2018). ML DDoS detection for consumer internet of things devices. *2018 IEEE Security and Privacy Workshops (SPW)*, 29–35.

7. Ferrag, M. A., Maglaras, L., Derhab, A., & Janicke, H. (2020). DL for cybersecurity: A review of recent advancements and future prospects. *Computers & Security, 92*, 101747.

8. Ferrag, M. A., Maglaras, L., & Ahmim, A. (2019). Privacy-preserving schemes for Ad Hoc Social Networks: A survey. *IEEE Communications Surveys & Tutorials, 21*(2), 1646–1686.

9. Garcia, S., Grill, M., Stiborek, J., & Zunino, A. (2014). An empirical comparison of botnet detection methods. *Computers & Security, 45*, 100–123.

10. Hodo, E., Bellekens, X., Hamilton, A., Dubouilh, P. L., Iorkyase, E., Tachtatzis, C., & Atkinson, R. (2016). Threat detection using artificial neural networks for IoT networks. *2016 International Symposium on Networks, Computers and Communications (ISNCC)*.

11. Idris, M., & Selamat, A. (2020). Hybrid IDS using signature-based and anomaly-based methods with AI techniques. *Journal of Intelligent & Fuzzy Systems, 39*(4), 5061–5071.

12. Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A DL approach for network IDS. *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 21–26.

13. Kim, G., Lee, S., & Kim, S. (2016). A novel hybrid intrusion detection method integrating anomaly detection with misuse detection. *Expert Systems with Applications, 41*(4), 1690–1700.

14. Kolosnjaji, B., Zarras, A., Webster, G., & Eckert, C. (2016). DL for classification of malware system call sequences. *Australasian Joint Conference on Artificial Intelligence*, 137–149.

15. Kwon, D., Kim, H., & Kim, H. K. (2021). Incremental learning of LSTM for cyber attack detection in the internet of things. *IEEE Access, 9*, 58673–58683.

16. Lashkari, A. H., Draper Gil, G., Mamun, M., & Ghorbani, A. A. (2017). Characterization of Tor traffic using time based features. *Proceedings of the 3rd International Conference on Information Systems Security and Privacy*, 253–262.

17. Li, F., Li, Y., & Lin, G. (2019). A survey of DL methods for cyber security. *Information, 10*(9), 1–30.

18. Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., & Lloret, J. (2017). Network traffic classifier with convolutional and recurrent neural networks for Internet of Things. *IEEE Access, 5*, 18042–18050.

19. Lyu, L., & Miao, H. (2023). Transformer-based detection system for encrypted malicious traffic. *Computers & Security, 125*, 102947.

20. Moustafa, N., Slay, J., Creech, G., & Turnbull, B. (2017). A holistic review of network anomaly detection systems: A comprehensive survey. *Journal of Network and Computer Applications, 102*, 63–82.

21. Nisioti, A., Mylonas, A., Yoo, P. D., & Katos, V. (2018). From intrusion detection to attacker attribution: A comprehensive survey of unsupervised methods. *IEEE Communications Surveys & Tutorials, 20*(4), 3369–3388.

22. Panigrahi, R., & Borah, M. D. (2020). A detailed analysis of CICIDS2017 dataset for designing an IDS. *International Journal of Engineering Research & Technology, 9*(2), 2278–2282.

23. Parwez, M. S., Rawat, D. B., & Garuba, M. (2017). Big data analytics for user-activity analysis and user-anomaly detection in mobile wireless network. *IEEE Transactions on Industrial Informatics, 13*(4), 2058–2065.

24. Rahman, M. A., Hossain, M. S., Muhammad, G., & Alhamid, M. F. (2019). Cybersecurity analysis in smart healthcare using DL and fog computing. *IEEE Access, 7*, 135911–135922.

25. Roopak, R., & Thomas, A. (2021). A hybrid model using deep neural networks for anomaly detection in IDS. *Journal of Intelligent Systems, 30*(1), 379–391.

26. Roy, S., Cheung, S., Sharma, P., & Goel, S. (2022). Enhancing IDS with ensemble learning and CICIDS2017 dataset. *Procedia Computer Science, 199*, 1500–1508.

27. Saied, A., Overill, R. E., & Radzik, T. (2016). Detection of known and unknown DDoS attacks using artificial neural networks. *Neurocomputing, 172*, 385–393.

28. Samrin, R. K., & Sulaiman, N. (2020). DL-based intrusion detection for the Internet of Things. *Journal of Intelligent & Fuzzy Systems, 38*(5), 5721–5731.

29. Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A DL approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence, 2*(1), 41–50.

30. Sittón-Candanedo, I., & Corchado, J. M. (2021). Cybersecurity threats detection in smart homes using DL. *Sensors, 21*(2), 591.

31. Sivaguru, R., & Rajendran, C. (2023). Lightweight IDS for edge computing using LSTM-CNN. *Computers & Security, 125*, 102948.

32. Sun, Y., Peng, S., & Guo, Y. (2019). Deep belief network for anomaly detection in industrial cyber-physical systems. *IEEE Transactions on Industrial Informatics, 15*(5), 3055–3063.

33. Tsai, C. F., Hsu, Y. F., Lin, C. Y., & Lin, W. Y. (2009). Intrusion detection by ML: A review. *Expert Systems with Applications, 36*(10), 11994–12000.

34. Vinayakumar, R., Soman, K. P., & Poornachandran, P. (2019). Evaluating DL approaches to characterize and classify network traffic. *Neural Computing and Applications, 32*, 1629–1650.

35. Wang, W., Zhu, M., Zeng, X., Ye, X., & Sheng, Y. (2017). Malware traffic classification using convolutional neural network for representation learning. *2017 International Conference on Information Networking (ICOIN)*, 712–717.